



***Asociación Profesional del Cuerpo Superior
de Sistemas y Tecnologías de la Información
de la Administración del Estado***

**Temas Específicos para la preparación de la Oposición al Cuerpo
Superior de Sistemas y Tecnologías de la Información de la
Administración del Estado.**

TEMAS ESPECÍFICOS III: Ingeniería de los Sistemas de Información

**Tema 92. Diseño arquitectónico de sistemas. Diagramas de
despliegue**

AUTOR: José Antonio Eusamio Mazagatos

Actualización 2017

ÍNDICE

1	INTRODUCCIÓN	3
1.1	CLASIFICACIÓN DEL DISEÑO DE PROGRAMAS	3
1.1.1	<i>Tareas a realizar en la fase de diseño</i>	4
1.1.2	<i>Actividades en la fase de diseño en Métrica V3</i>	5
1.2	ARQUITECTURAS DE SISTEMAS MÁS COMUNES.....	6
1.2.1	<i>Estilos Arquitectónicos</i>	6
1.2.2	<i>Arquitectura centrada en datos</i>	8
1.2.3	<i>Arquitectura centrada en Flujo de datos.....</i>	10
1.2.4	<i>Arquitectura de llamada y retorno</i>	12
1.2.5	<i>Arquitecturas de Objetos Distribuidas.....</i>	14
1.2.6	<i>Arquitecturas orientadas a Objetos</i>	20
1.2.7	<i>Arquitectura MVC.....</i>	26
1.2.8	<i>Arquitecturas en capas.....</i>	28
1.2.9	<i>Arquitecturas de sistemas de Tiempo Real</i>	29
1.2.10	<i>Sistemas Transaccionales.....</i>	32
2	EL LENGUAJE DE MODELADO UNIFICADO (UML)	35
2.1	EVOLUCIÓN DE UML DESDE OTRAS METODOLOGÍAS/NOTACIONES OO	35
2.2	UML 1.X	36
2.3	UML 2.X	37
2.3.1	<i>Estándares que conforman UML 2.X.....</i>	37
2.3.2	<i>Tipos de diagramas (UML 2.5)</i>	38
2.3.3	<i>Arquitectura de Sistemas</i>	40
2.4	DIAGRAMA DE DESPLIEGUE	41
3	GLOSARIO.....	46

1 Introducción

En cualquier proyecto de Ingeniería del Software se debe seguir una metodología de desarrollo y se emplearán distintas técnicas y herramientas. Dicho proyecto contará con las fases de Definición de requisitos, Análisis, Diseño, Verificación y Validación (Pruebas de Calidad del Software), Pruebas y Mantenimiento.

Este tema se centrará en el apartado del Diseño, distinguiendo entre los dos grandes tipos existentes como son el Diseño Orientado a Objetos y el Diseño Estructurado.

Así pues, se define **Diseño del Sistema**, según Glosario de Términos de Calidad e Ingeniería del Software de la Asociación Española de Control de Calidad, como “el proceso de definición de la arquitectura software: componentes, módulos, interfaces, procedimientos de prueba y datos de un sistema que se crean para satisfacer unos requisitos especificados”.

Según la Metodología Métrica V3, el objetivo del proceso de Diseño del Sistema de Información (DSI), es la definición de la arquitectura del sistema y del entorno tecnológico que le va a dar soporte, junto con la especificación detallada de los componentes del sistema de información. Distingue entre diseño orientado a objetos y estructurado.

En términos generales, el **diseño** se puede definir como: ***"El proceso de aplicar distintas técnicas y principios con el propósito de definir un dispositivo, proceso o sistema, con los suficientes detalles como para permitir su implementación física"***.

El **diseño del software** es un proceso aplicado desde hace no más de 3-4 décadas. Se trata de garantizar la calidad de las aplicaciones en sentido industrial, es decir, tanto para asegurar que cumplen una serie de criterios y normas medibles como de garantizar que cumplen las necesidades requeridas por el cliente o usuario.

El hecho de que el coste de mantenimiento es mucho mayor que el de desarrollo lleva a la conclusión de que el esfuerzo requerido para la corrección de un error es varias veces mayor si debe efectuarse en una fase tardía del desarrollo (incluyendo mantenimiento y soporte).

El **diseño modular** permite que se integren elementos fiables y probados que permitan la máxima flexibilidad en cuanto a intercambio de componentes y faciliten aislar los errores en componentes determinados.

El **diseño estructurado** amplía el planteamiento modular haciendo que la estructura del diseño se plasme en reglas y objetivos que veremos a continuación.

1.1 Clasificación del diseño de programas

La Asociación Española de Control de Calidad define:

- ✓ El **diseño de alto nivel** como el proceso de definición de componentes y sus interfaces para establecer la estructura del Sistema.
- ✓ El **diseño detallado** es el proceso de refinar y expandir el diseño de alto nivel para que contenga descripciones más detalladas de la lógica del proceso, las estructuras de datos y las definiciones de datos.

De forma general, y para concretar más las definiciones anteriores, se pueden destacar las siguientes categorías de diseño:

- ✓ **Arquitectura general del sistema:** Este primer paso considerará al sistema como un todo (sw, hw y rrhh). Es de alto nivel. Es un diseño externo porque plantea las interfaces del sistema con el exterior (podremos tomar datos de la fase de análisis o incluso EVS-AGS).
- ✓ **Diseño de alto nivel o arquitectónico o estructurado:** Desarrolla una estructura modular de programa y representa las relaciones de control entre los módulos (Clases principales en Orientación a Objetos).
- ✓ **Diseño de bajo nivel o detallado:** Realizado a partir del de alto nivel que, en el caso de un software se puede detallar en:
 - ✓ **Diseño de datos:** Tanto lógico como físico (por ejemplo, Modelo Lógico Relacional y la implantación en un SGBDR concreto).
 - ✓ **Diseño procedimental:** Transforma los elementos estructurales en una descripción de los elementos del software. El diseño de procedimientos se realiza después de que se ha establecido la estructura del programa y de los datos. Define los algoritmos de procesamiento necesarios. Debe especificar los detalles de los procedimientos sin ambigüedad. En Orientación a Objetos nos referiríamos al detalle de los métodos.

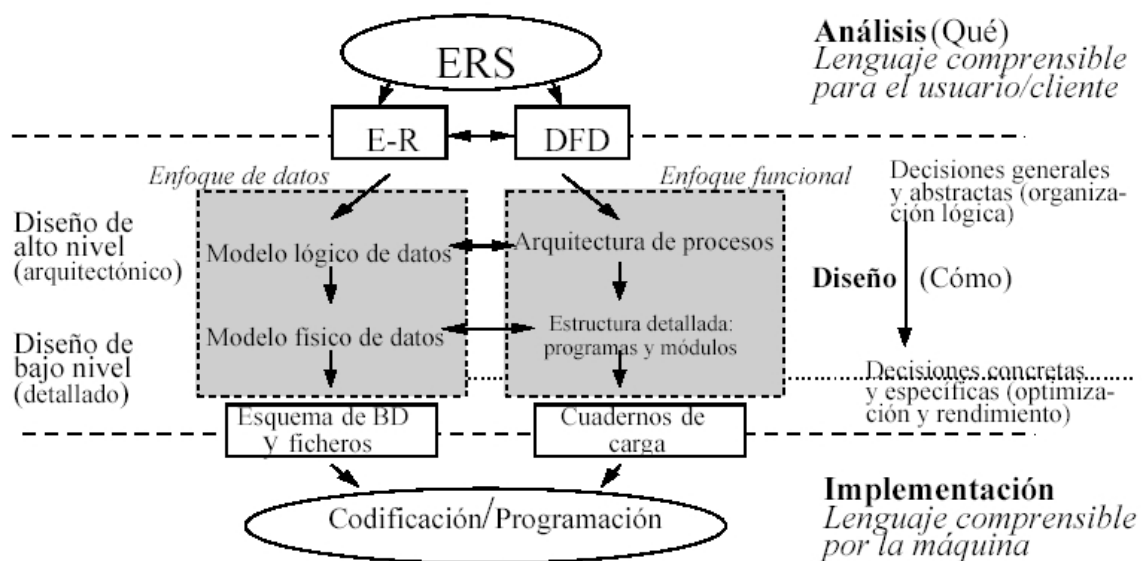


Figura 1.- Tareas del Diseño de Alto nivel y Diseño detallado en relación al análisis e Implementación de un sistema SW

1.1.1 Tareas a realizar en la fase de diseño

Entre las tareas a realizar en la fase de diseño del sistema de información reconocidas por la mayoría de autores de Ingeniería del Software se encuentran:

Relativas al diseño de “alto nivel”:

- ✓ **Selección de la arquitectura global del sistema.** O bien viene impuesta por el Plan de Sistemas donde esté encuadrado el proyecto o porque hay similitud con soluciones ya adoptadas.
- ✓ **Descomposición en subsistemas de sistemas.** Tratamos aquí subsistemas físicos y no lógicos como en la fase de análisis.
- ✓ **Identificación de concurrencias.** Se trata de buscar un compromiso que atienda a las limitaciones del hardware que estamos considerando, al rendimiento que esperamos para nuestras aplicaciones y al coste del sistema en general.
- ✓ **Control de los recursos compartidos.** Hay que considerar controles sobre el acceso al hardware y sobre el acceso a las bases de datos (p.e.). Todos los recursos globales del sistema tienen que gestionarse de forma que se garantice el acceso a los mismos por todas aquellas entidades autorizadas.
- ✓ **Gestión de los datos.** Elección del diseño físico más adecuado para almacenar los datos del sistema.

- ✓ **Diseño de las interfaces.** Entre los distintos componentes que forman el sistema (sw-hw, sw-sw, etc)

Relativas al diseño de “bajo nivel”:

- ✓ **Identificación y manejo de excepciones.** En la fase de análisis se responde a lo que se quiere que el sistema haga. Aquí se pretende completar el análisis y describir lo que el sistema no tienen que hacer y también qué acciones hay que tomar cuando se presenta una excepción.
- ✓ **Selección de los algoritmos y de las estructuras de datos.** Aquí nos referimos al diseño procedimental o diseño de bajo nivel, tal y como reflejaba la Figura 1.
- ✓ **Preparación para la migración y carga inicial de datos.**
- ✓ **Especificación técnica de las pruebas.**

1.1.2 Actividades en la fase de diseño en Métrica V3

A modo de referencia, y sin ánimo de ser exhaustivo, ya que Métrica V3 está tratada en profundidad en otro tema del temario, se incluyen las actividades que forman parte del proceso de Diseño del Sistema de información.

- ✓ **DSI1: Definición de la Arquitectura del Sistema.**
 - ✓ Especificación de las distintas particiones físicas del mismo, la descomposición lógica en subsistemas de diseño y la ubicación de cada subsistema en cada partición, así como la especificación detallada de la infraestructura tecnológica necesaria para dar soporte al SI
- ✓ **DSI2: Diseño de la Arquitectura de Soporte.**
 - ✓ Comprende el diseño de los subsistemas de soporte identificados en la actividad DSI1 y la determinación de los mecanismos genéricos de diseño. Estos sirven de guía en la utilización de diferentes estilos de diseño, tanto en el ámbito global del SI como en el diseño de detalle.
- ✓ **DSI3: Diseño de Casos de Uso reales.**
- ✓ **DSI4: Diseño de Clases.**
- ✓ **DSI5: Diseño de la Arquitectura de módulos del sistema.**
- ✓ **DSI6: Diseño físico de datos.**
- ✓ **DSI7: Verificación y aceptación de la arquitectura del sistema.**
- ✓ **DSI8: Generación de especificaciones de construcción.**
 - ✓ Construcción del sistema a partir de componentes (unidades básicas de construcción).
- ✓ **DSI9: Diseño de la migración y carga inicial de datos.**
- ✓ **DSI10: Especificación técnica del Plan de Pruebas.**
- ✓ **DSI11: Establecimiento de requisitos de implantación.**
 - ✓ Documentación para usuario, formación, infraestructura, utilización, ...
- ✓ **DSI12: Aprobación del diseño del sistema de información.**

El orden y la relación entre actividades, así como si son del paradigma estructurado u orientado a objetos se recoge en la figura.

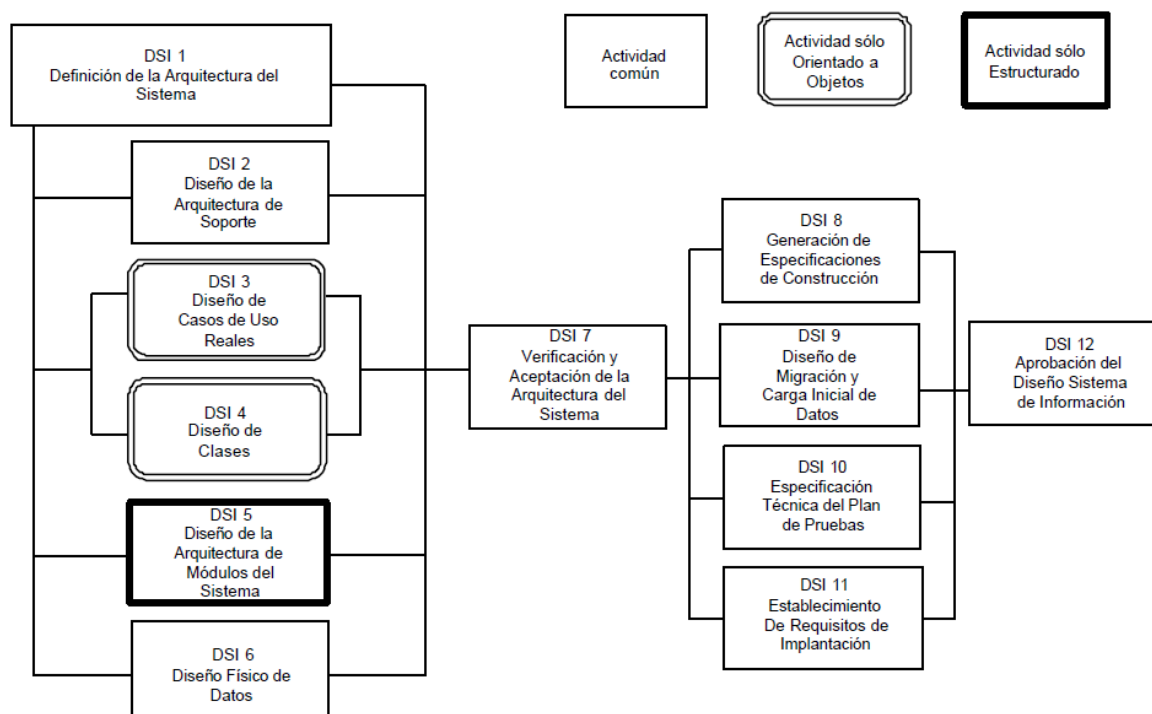


Figura 2 .- Orden y relación entre actividades del proceso de Diseño del Sistema de información en MetricaV3

1.2 Arquitecturas de Sistemas más Comunes.

Todo sistema software tiene una arquitectura. En algunos casos, la arquitectura es intencional, mientras que en otros es accidental. La mayoría de las veces es una mezcla de ambas a la vez, nacida de las consecuencias de múltiples decisiones de diseño tomadas por sus arquitectos y sus desarrolladores durante la vida de un sistema, desde su inicio hasta su evolución. En este sentido, la arquitectura de un sistema consistiría en la identificación de las decisiones de diseño más importantes que dan forma a un sistema, donde se mide significativamente el coste del cambio y el impacto sobre el uso.

Desde esta perspectiva, toda la arquitectura forma parte del diseño, pero no todo el diseño es arquitectura. Las decisiones arquitectónicas son a menudo afirmadas como patrones que se entrelazan a través de las abstracciones en un sistema, dando lugar a texturas que trascienden el código ejecutable en bruto. En el nivel más alto de abstracción, podemos discernir patrones que dan a cada sistema un estilo arquitectónico particular. Para todos los sistemas de suficiente complejidad, sólo se puede entender y razonar sobre la arquitectura de un sistema desde múltiples puntos de vista, cada uno de los cuales representa las preocupaciones de un determinado conjunto de partes interesadas. El reto del proceso de ingeniería que hace que cualquier sistema software se manifieste es, por lo tanto, desarrollar un conjunto de decisiones arquitectónicas que produzcan un equilibrio óptimo de las fuerzas funcionales y no funcionales que pesan sobre cada una de estas vistas.

Un aspecto muy importante en arquitectura del software son los patrones tanto de diseño como de arquitectura. Los complementan los estilos arquitectónicos. Ambos se preocupan de los stakeholders. Los estilos favorecen al tratamiento estructural, la investigación, la teoría, mientras que los patrones están más cerca del diseño, la práctica, el proceso y el código. Los estilos estarían más cerca del diseño arquitectónico mientras que los patrones más del diseño detallado. El tema de patrones se trata en más profundidad en el tema 89.

1.2.1 Estilos Arquitectónicos

Un estilo describe una clase de arquitectura o piezas de la misma. Los estilos se pueden reutilizar en situaciones semejantes en el futuro. Desde que surgió la arquitectura del software se generalizó la

arquitectura cliente – servidor, posteriormente entró en auge las arquitecturas en capas y las basadas en componentes y más recientemente las basadas en recursos y servicios. Todas ellas se tratan en más detalle en este tema o en otros temas del temario.

Cada estilo describe una categoría de sistemas que incluye:

1. Un conjunto de componentes (una base de datos o módulos de Negocio/Operación) que realizan una función requerida por el sistema.
2. Un conjunto de conectores que permiten la comunicación, coordinación y cooperación entre los componentes.
3. Restricciones que definen como se integran los componentes para formar el sistema.
4. Modelos semánticos que permiten que un diseñador entienda las propiedades generales del sistema al analizar las propiedades conocidas de sus partes.

Cuando definimos una arquitectura, la tipificamos y la distinguimos, pero existen preguntas generales que se plantean y que conviene tener en cuenta:

- ✓ ¿Cuáles son las arquitecturas alternativas?
- ✓ ¿La arquitectura elegida abarca todo un contexto?
- ✓ ¿La arquitectura elegida es una variante incluida en un conjunto más amplio?
- ✓ ¿En qué magnitud contrasta con las otras alternativas?
- ✓ ¿Cuántas formas podría adoptar?

No solo es una cuestión clasificatoria. Optar por una forma de arquitectura tiene un valor distintivo. Los estilos se manifiestan en la arquitectura teórica descriptiva; los partidarios de los estilos se definen como arquitectos. Los que se agrupan en torno a patrones se confunden con ingenieros o diseñadores, incluso con programadores sistemáticos o analistas de software. Ambos con mayor o menor grado participan abarcando la Arquitectura del Software. Los estilos se encuentran en el centro de la arquitectura, constituyendo su esencia. Los patrones de arquitectura están claramente dentro de la disciplina, no así los patrones de diseño.

Se establece que una Arquitectura Software se define mediante tres elementos, **elementos, forma y razón**.

Los elementos son de tres tipos:

- ✓ **procesamiento**: Suministra la transformación de los datos (Lógica de Negocio)
- ✓ **datos**: Contiene la información a procesar (Core Business)
- ✓ **conexión**: Se encarga de las llamadas a procedimientos, mensajes, etc.

La forma determina las propiedades y relaciones entre los elementos mientras que la razón es una justificación de la motivación para la elección de elementos y sus formas. Por tanto un estilo arquitectónico es una abstracción de tipos de elementos y aspectos formales a partir de arquitecturas específicas. Es decir, al conjunto de reglas de diseño que identifica las clases de componentes y conectores que se pueden utilizar para componer el sistema o subsistema junto con las restricciones locales o globales de la forma en que la composición se lleva a cabo mediante componentes y conectores:

- ✓ **Componentes**: Engloba a los sistemas encapsulados indicando la forma de empaquetado y la forma en que interactúan con otros componentes.
- ✓ **Conectores**: Se refiere a como los procesos interactúan por medio de protocolos de transferencia de mensajes o por flujo de datos, etc.

Estilo arquitectónico es una descripción del patrón de los datos y la interacción de control entre los componentes, ligada a una descripción informal de los beneficios e inconvenientes del uso del estilo. Los estilos son entidades que ocurren en un nivel abstracto y puramente arquitectónico.

La mayoría de los problemas se pueden resolver utilizando alguna de los estilos arquitectónicos o arquitecturas que mencionaremos a continuación, o bien una combinación de ellas. Se tratarán en profundidad a lo largo del tema las más importantes o conocidas. En el caso de que estas arquitecturas se traten en otros temas se referirá a los mismos.

Arquitectura centrada en datos (arquitecturas de pizarra, Data Warehouse)

Arquitectura de flujo de datos: (Tuberías y filtros, proceso secuencial en lotes)

- ✓ **Transformación por lotes:** Una transformación por lotes o “batch” es una transformación secuencial de una entrada en una salida sin ningún tipo de interacción con el exterior.
- ✓ **Transformación continua:** Una transformación continua es un sistema en el que las salidas dependen activamente de entradas cambiantes y deben ser periódicamente actualizadas.

Arquitecturas de llamada y retorno (llamadas a procedimientos remotos, Modelo Vista Controlador)

- ✓ **Interfaz interactiva:** Este tipo de sistemas está dominado por la interacción entre el sistema y agentes exteriores independientes de éste.
- ✓ **Arquitecturas estratificadas o por capas o niveles.**
- ✓ **MVC - Modelo Vista Controlador**

Arquitecturas orientadas a objetos:

Se trata en detalle el paradigma orientado a objetos en este tema.

Arquitecturas orientadas a aspecto:

El desarrollo de este paradigma se detallará en el tema 89.

Arquitecturas orientadas a servicios (SOA)

El desarrollo de estas arquitecturas se realiza en el tema 55 “*El procesamiento cooperativo y la arquitectura cliente-servidor. Arquitectura SOA*”.

Arquitecturas de Sistemas Especiales:

- ✓ **Sistema en tiempo real:** Un sistema en tiempo real es un sistema interactivo con fuertes restricciones de tiempo. El sistema debe garantizar la respuesta dentro de un intervalo de tiempo cerrado (no necesariamente corto, pero sí estricto en su cumplimiento). Por ejemplo 25 imágenes por segundo, Tiempo de respuesta Inferior a “X” msg
Estos sistemas son complejos, conllevan un control exhaustivo de las interrupciones y la coordinación de las distintas C.P.U's.
- ✓ **Sistemas transaccionales:** La mayoría de estos sistemas transaccionales trabajan con múltiples usuarios y utilizan concurrencia. Cada transacción debe ser manejada como una entidad sin relación con las otras transacciones.

Es de resaltar que en un sistema completo y complejo tendremos una mezcla de estilos diferentes para cada parte del sistema, en distinto nivel de detalle y profundidad, siendo la más predominante o importante para el negocio la que le otorgará el estilo final.

1.2.2 Arquitectura centrada en datos

Este estilo se fundamenta en el almacenamiento de datos (archivos o bases de datos) al que acceden otros componentes que realizan acciones sobre los datos. El cliente accede al repositorio central. Existen dos modos fundamentalmente, pasivo o en Pizarra:

- ✓ **Repositorio pasivo:** El software cliente accede a los datos independientemente de los cambios de los datos o de las acciones del software de otro cliente.
- ✓ **Repositorio pizarrón:** el repositorio envía notificaciones a los clientes cuando los datos de su interés cambian, siendo por tanto un ente activo.

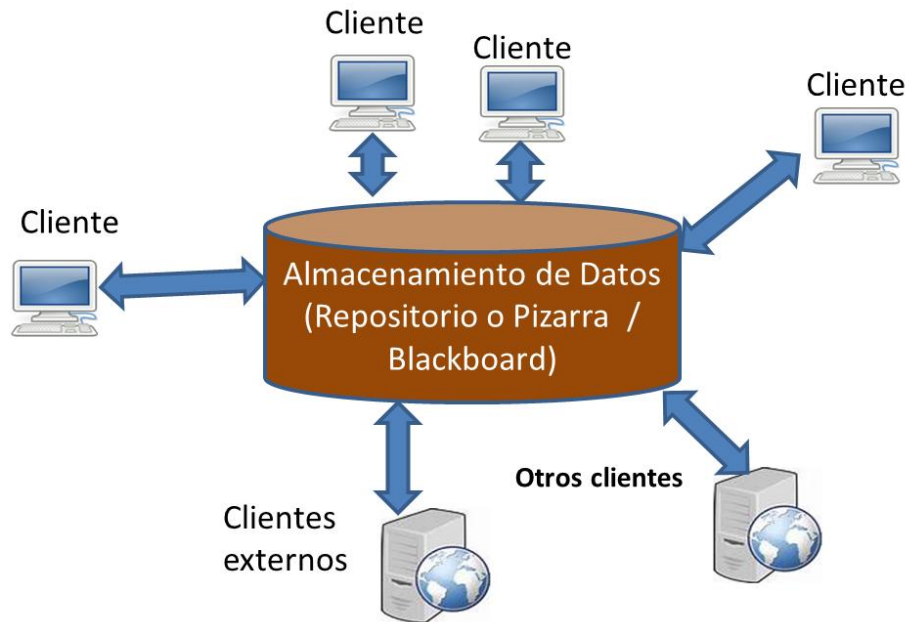


Figura 3 .- Arquitectura centrada en los datos

1.2.2.1 Arquitectura en pizarra

La mejor manera de explicar este estilo es imaginar un número de estudiantes que están escribiendo en la pizarra al mismo tiempo tratando de resolver un problema, en el que el maestro primero le pregunta a cada estudiante lo que quiere escribir en la pizarra y decide qué estudiante tiene la mejor idea. Y cuando el estudiante haya terminado, el proceso se repite. Esta es la idea detrás de la arquitectura en pizarra. Los estudiantes recibirían el nombre de **“fuentes de conocimiento”**. El profesor se denomina **“intérprete de comandos de control”**. En la Arquitectura en Pizarra varios subsistemas especializados pueden reunir sus conocimientos para construir una posible solución parcial o aproximada.

La idea detrás de la Arquitectura de Pizarra es una colección de programas independientes que trabajan conjuntamente de manera colaborativa en una estructura de datos común. Cada programa especializado trabaja para la solución de una parte específica de la tarea general, y todos los programas trabajan juntos en la solución total.

La Arquitectura en pizarra consta de tres componentes tal y como se ven en la siguiente figura:

- ✓ **Fuente de conocimiento.** La fuente de conocimiento es un componente que se añade a la solución del problema. Puede ser cualquier cosa que se lee de un cierto nivel de pizarra, y propone una modificación a los componentes de la pizarra.
- ✓ **Pizarra.** Es la estructura de datos común de las fuentes de conocimiento. Es capaz de representar a todos los estados de algún espacio del problema. Contiene varios niveles de descripción con respecto al espacio del problema.
- ✓ **Intérprete de comandos de control.** Determina que fuente de conocimiento tiene la oportunidad de cambiar la pizarra. En cada ejecución, identifica los cambios a la pizarra, activa las fuentes de conocimiento apropiadas, selecciona una de estas y la ejecuta.

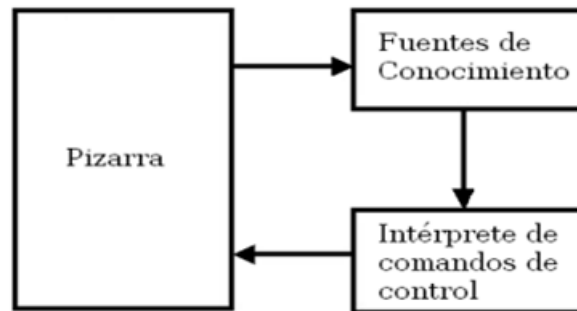


Figura 4 .- Flujo de control en una Arquitectura centrada en los datos del tipo Pizarra

Un ejemplo de arquitectura centrada en datos sería un Datawarehouse, que obtiene los datos de distintas fuentes mediante procesos de extracción y transformación (ETL) para adecuarlos al modelo de datos del sistema.

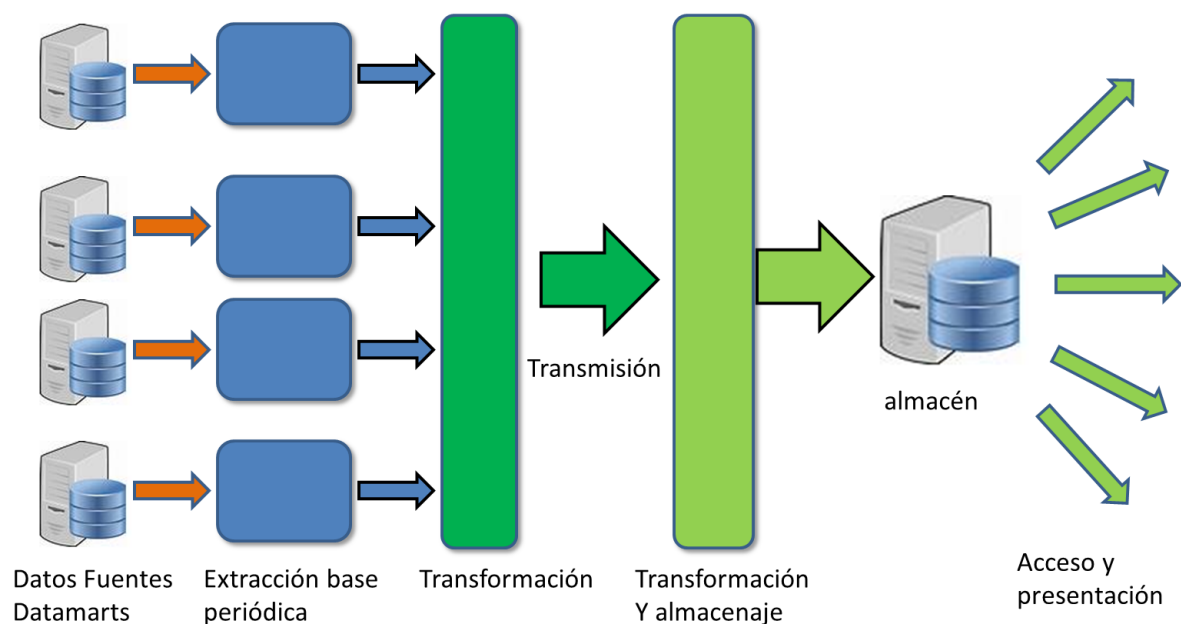


Figura 5 .- Flujo de control en una Arquitectura centrada en los datos DataWarehouse

1.2.3 Arquitectura centrada en Flujo de datos

Este estilo de arquitectura enfatiza la reutilización y la modificabilidad. Es adecuada para sistemas con transformaciones de datos en pasos sucesivos (cuando los datos de entrada se transforman en datos de salida mediante una serie de componentes para el cálculo o la manipulación).

Ejemplos de este estilo son las arquitecturas de tuberías y filtros y las de proceso secuencial en lotes (batch).

1.2.3.1 Tuberías y filtros

Una tubería (pipeline) es una arquitectura que conecta componentes computacionales (filtros) a través de los conectores (pipes) y las comunicaciones se ejecutan como un flujo. Recuerda mucho al proceso de análisis de la técnica de Diagrama de Flujo de Datos y se relaciona con la técnica de diseño de estructura de cuadros de Constantine, que se describe en el tema 89.

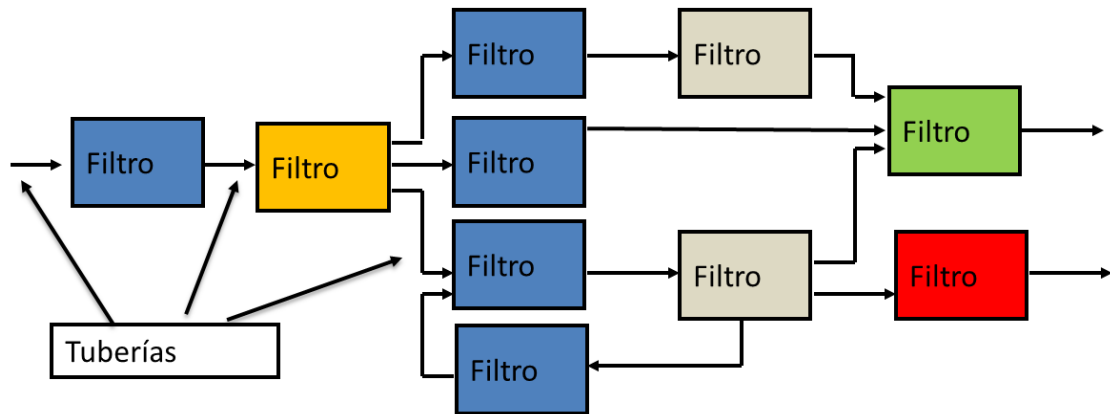


Figura 6 .- Arquitectura centrada en flujo de datos: Tuberías y filtros

Los filtros no necesariamente realizan únicamente tareas de filtrado sino que ejecutan formas variables de transformación. Está diseñado para esperar la entrada de datos con cierta forma y producir su salida (al siguiente filtro) de una forma específica. No es necesario que el filtro conozca el funcionamiento de los filtros vecinos.

Un ejemplo típico es la gestión de procesos en Unix.

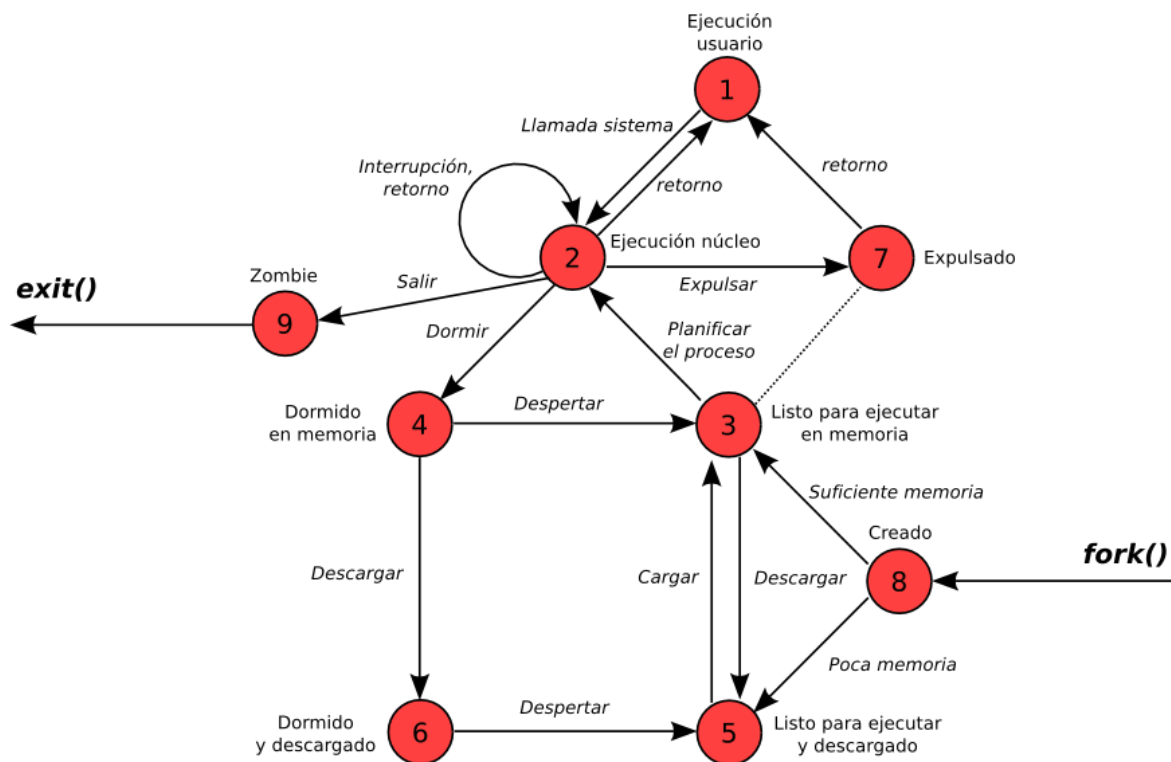


Figura 7 .- Ejemplo de Arquitectura de Tuberías y filtros - proceso fork en Unix

Otro ejemplo claro sería un Compilador, con un flujo compuesto por las siguientes acciones, y cuyo flujo se muestra en la figura siguiente:

- ✓ Código fuente
- ✓ Análisis léxico
- ✓ Análisis Sintáctico
- ✓ Análisis semántico
- ✓ Manejador de errores
- ✓ Generación de Código Intermedio

- ✓ Optimización
- ✓ Generación de Código Objeto

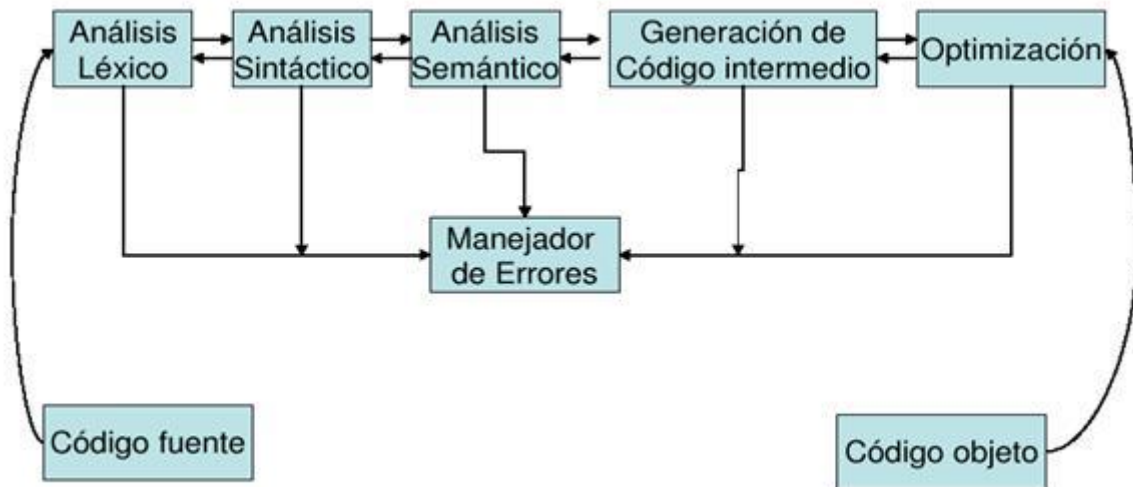


Figura 8 .- Ejemplo de Arquitectura de Tuberías y filtros – Flujo de acciones de un Compilador

Este estilo se podría usar cuando:

- ✓ Se puede especificar la secuencia de un número conocido de pasos
- ✓ No se requiere esperar la respuesta asincrónica de cada paso
- ✓ Se busca que todos los componentes sean capaces de inspeccionar y actuar sobre los datos que vienen de la entrada según una precedencia fijada y no al revés.

Entre las principales ventajas de este estilo arquitectónico se encuentran:

- ✓ Simple de entender e implementar
- ✓ Facilidad de mantenimiento y mejora
- ✓ Facilidad de diagnóstico en el caso de errores
- ✓ Fuerza un procesamiento secuencial aunque permite una ejecución concurrente

Conviene señalar no obstante las desventajas del estilo, que es demasiado simplista ya que entre otras cosas no maneja explícitamente condicionales, bucles y otras lógicas de control de flujo. Se podría decir que no es muy apto para situaciones interactivas y podría ocasionar problemas de rendimiento ya que implica que los datos se transmiten de forma completa.

1.2.3.2 Estilo secuencial por lotes

En el estilo secuencial por lotes (batch sequential) los componentes son programas independientes. Cada paso se ejecuta hasta completarse antes que se inicie el paso siguiente. Muy adecuado para tareas independientes, in intervención manual y repetitivas. Se puede considerar una simplificación del anterior.

1.2.4 Arquitectura de llamada y retorno

Con esta arquitectura se obtiene una estructura de programa que resulta relativamente fácil de modificar y cambiar de tamaño. Existen dos subestilos:

- ✓ Arquitectura de programa principal/subprograma: Esta estructura de programa clásica separa la función en una jerarquía de control donde un programa “principal” invoca a varios componentes de programa, que a su vez pueden invocar a otros componentes.
- ✓ Arquitectura de llamada a procedimiento remoto: Los componentes de una arquitectura de programa principal/subprograma se distribuyen entre varios ordenadores o sistemas en red.

1.2.4.1 Arquitectura de programa principal/subprograma

La programación procedimental precede a la programación orientada a objetos. La llamada a procedimiento convencional es una llamada en el mismo sistema en el que se invoca y se llama a un procedimiento local. Un procedimiento o función es una estructura de control que proporciona la abstracción a una acción. La acción de una función se invoca a través de una llamada a función.

Para usar diferentes variables una llamada a función debe tener argumentos. El valor o referencia de cada argumento se pasa a la función.

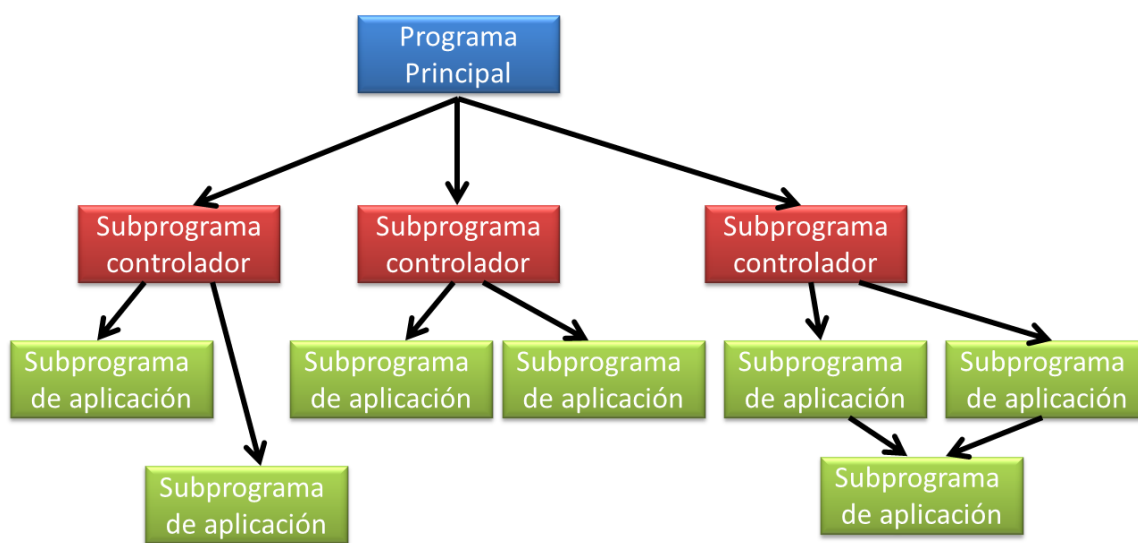


Figura 9 .- Arquitectura de programa principal/subprograma

Este estilo arquitectónico es tratado en profundidad por la técnica de diseño de estructura de cuadros de Constantine, que se describe en el tema 89 de técnicas de diseño.

1.2.4.2 Arquitectura de llamada a procedimiento remoto

El objetivo de la llamadas a procedimiento remoto (Remote Procedure Call) RPC por sus siglas en inglés es extender a los sistemas distribuidos el mecanismo de llamadas a procedimientos y subrutinas de los lenguajes de programación. RPC está basado en el modelo cliente – servidor.

RPC permite que un proceso localizado en una máquina pueda ejecutar código localizado en otra máquina remota, sin preocuparse por la comunicación entre ambas. En el modelo de llamada a procedimiento remoto, un proceso realiza una llamada a procedimiento de otro proceso que posiblemente esté localizado remotamente.

- ✓ Los datos se pasan a través de argumentos.
- ✓ Cuando un proceso recibe una llamada, se ejecuta la acción codificada en el procedimiento.
- ✓ Se notifica la finalización de la llamada al proceso que lo invoca.
- ✓ Si existe un valor de retorno se envía desde el proceso invocado.

Este estilo arquitectónico se ha utilizado por ejemplo en la arquitectura de objetos remotos, CORBA que se describe a continuación.

En la Máquina cliente está el Cliente y el extremo del cliente (stub).

En la Máquina servidor está el extremo del servidor (skeleton) y el servidor.

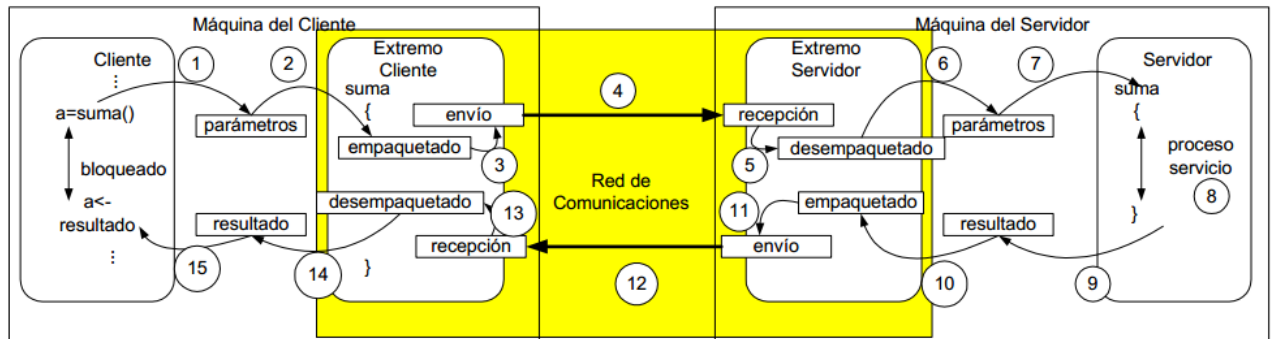


Figura 10 .- Flujo de acciones en una Arquitectura de llamada a procedimiento remoto

En la llamada a procedimiento remoto todos los parámetros que se utilicen en las llamadas han de ser pasados por valor; y no por referencia, ya que ambos procedimientos se ejecutan en espacios de memoria diferentes. Para que estos parámetros puedan ser enviados a través de la red, es necesario que sean serializables, es decir, convertibles en una corriente de bytes.

1.2.5 Arquitecturas de Objetos Distribuidas

Actualmente la computación presenta como principal problema el hecho de que no es práctico desarrollar aplicaciones monolíticas ya que muchos de los programas desarrollados en los actuales lenguajes de programación son demasiado grandes. La idea es romper el problema en muchos subproblemas menores y crear soluciones a cada sub-problema por separado. Estas soluciones funcionan bien si se construyen en interfaces estándar, pero el problema de crear una interfaz estándar es un “problema de interoperabilidad”.

Partiendo de esta idea puede pensarse en sistemas distribuidos. Éstos son sistemas en el cual todas las funciones de la aplicación se ofrecen como objetos, de forma que cada uno de estos objetos puede hacer uso de todos los métodos ofrecidos por otros objetos del mismo sistema o de otro. En estos sistemas distribuidos es muy importante la definición de la interfaz entre los módulos (objetos), es decir del protocolo de comunicación entre ellos, ya que esta interfaz específica a los demás módulos que servicios ofrece y como se usan. Presentan las siguientes características:

- ✓ Concurrencia. Característica por la que múltiples (dos o más) objetos, recursos, etc..., coexisten en el tiempo de ejecución, en el caso de la programación distribuida pueden ser utilizados o consumidos simultáneamente por los objetos con los que interactúan.
- ✓ Independencia ante Fallos o Mayor resistencia ante fallos. Al estar distribuidos sobre distintos recursos, si un objeto del sistema distribuido falla, lo hace independientemente del resto al utilizar recursos físicos distintos. Por tanto no condiciona la ejecución del resto, lo que permite que el resto de componentes puedan continuar ejecutando sus acciones normalmente. Aunque una parte del sistema falle, el sistema en su conjunto continua (o puede seguir) trabajando.

Como se ha mencionado anteriormente, se tiene la necesidad de emplear estándares en los sistemas distribuidos. En primer lugar, una de las razones es la diversidad: los seres humanos trabajan en diferentes lugares y el almacenamiento de la información se realiza en diferentes sitios. Para lograr respuestas rápidas e independencia, es necesario colocar a las computadoras cerca de las personas que las usan, lo que lleva al concepto de computación personal. Sin embargo, las personas requieren de compartir información y procesos, por tanto los sistemas distribuidos se tienen que adaptar a diferentes requerimientos operacionales. Por ejemplo, la automatización de una fábrica requiere

respuestas en tiempo real y un alto grado de confiabilidad, mientras que las aplicaciones en una oficina requieren alta seguridad en lugar de respuestas en tiempo real. Otra de las razones es la fragmentación: hasta ahora, no existe una arquitectura abierta que facilite la construcción de sistemas distribuidos en una base de diversos vendedores, la cual pueda expandirse a diversos dominios de aplicación.

Como se ha expuesto en el punto anterior, las llamadas a procedimientos remotos (RPC, Remote Procedure Call) son un mecanismo básico en el desarrollo de aplicaciones en red para los sistemas distribuidos. La extensión de estos mecanismos de RPC a una programación orientada a objetos dio lugar a los modelos de objetos distribuidos. En este procedimiento operativo de RPC se fundamentan tecnologías tan significativas en la interconexión de objetos distribuidos como RMI o CORBA.

Este nuevo tipo de arquitectura de objetos distribuida que se va a presentar es una arquitectura abierta, ya que permite que se vayan agregando nuevos recursos conforme vayan siendo requeridos. Aporta la posibilidad de desarrollar un sistema flexible y escalable, que es posible reconfigurarlo dinámicamente mediante la inclusión de nuevos objetos. Se presenta un modelo de objetos distribuidos que describe los aspectos del paradigma de objetos que es aceptado por la tecnología, tales como la herencia, los interfaces, las excepciones, o el polimorfismo, entre otros.

Dando una visión general, en una arquitectura de objetos distribuidos cada entidad distribuida es un objeto que provee y recibe servicios a otros objetos. Los objetos pueden ser distribuidos entre un número de computadoras en una red y comunicarse a través de una capa intermedia llamada middleware. Este middleware actúa como un bus de software que proporciona un conjunto de servicios que permiten a los objetos comunicarse, y que recibe el nombre de ORB (objeto intermediario de solicitudes).

A lo largo de este punto se presentarán las distintas tecnologías tales como RMI, como mecanismo de comunicación entre objetos remotos Java y, por tanto, como tecnología de creación de aplicaciones Java distribuidas mediante llamadas a procedimientos remotos. Asimismo, en este mismo punto se analizará la arquitectura de objetos distribuidos del grupo OMG. En este escenario, se describe CORBA como arquitectura de comunicación entre objetos remotos heterogéneos y, por consiguiente, como tecnología de desarrollo de aplicaciones distribuidas, mediante llamadas a métodos remotos e independientemente del lenguaje de implementación de los objetos clientes y servidores, de la ubicación del correspondiente servicio, sistema operativo, etc.

1.2.5.1 Elementos de las arquitecturas de objetos distribuidos

1.2.5.1.1 Objetos distribuidos

Los elementos que interactúan en estas arquitecturas son los objetos distribuidos. Un objeto distribuido es aquel que puede ser llamado desde un sistema remoto por otro objeto mediante la utilización de un método de invocación remota. Este método se invoca mediante un mensaje, el cual hará que se ejecute el método, y el resultado se devuelve mediante otro mensaje. Estos presentan como ventaja el hecho de que los métodos remotos están asociados a objetos remotos. Esta nueva forma de desarrollar es más natural para el desarrollo orientado a objetos y admite modelos de programación orientada a eventos.

Como se ha comentado anteriormente, en este tipo de arquitecturas existe una capa intermedia denominada middleware. Los objetos distribuidos hacen uso de esta capa, obteniendo así un nivel de abstracción para la comunicación de los objetos distribuidos. De la misma manera el Middleware oculta la localización de objetos, los protocolos de comunicación, el hardware de computadora y los sistemas Operativo, de manera que supone una menor carga en la comunicación.

1.2.5.1.2 Middleware ORB (Object Request Broker)

Es el elemento central de la arquitectura. Proporciona las funcionalidades de interconexión entre los objetos distribuidos y representa un bus de comunicación entre objetos. Cualquier comunicación entre dos objetos cualesquiera de una aplicación se realiza por medio de este middleware, ya que éste

sabe de todos los objetos que hay en el sistema y de sus interfaces. El escenario de aplicación elemental es:

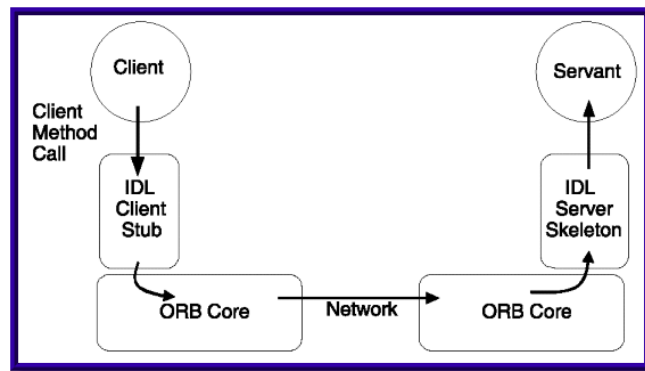


Figura 11 .- Comunicación Cliente-servidor entre objetos distribuidos ORB

En estas arquitecturas existe un lenguaje mediante el cual se describen los métodos que un determinado objeto del entorno proporciona al resto de elementos del mismo. Este lenguaje recibe el nombre de **IDL** (Interface Definition Language). El uso de este IDL es interesante porque cada aplicación es heterogénea, de modo que así se consigue una forma estándar de definir interfaces a los objetos. Igualmente debe realizarse el proceso inverso, es decir, un mapeo de lenguajes que traduzca la definición IDL a un lenguaje de programación. Esto se consigue haciendo uso de lo que se denominan **Stub** y **Skeleton**, que representan el código del cliente y del servidor respectivamente.

El Stub es el código cliente generado en base a la definición IDL que está asociado al objeto remoto con el que se desea interactuar. Simula para el cliente los métodos del objeto remoto, asociando a cada uno de los métodos una serie de funciones que realizan la comunicación con el objeto servidor. Contiene las llamadas para realizar el proceso de traducción desde el lenguaje de programación elegido a un formato intermedio y pedir al ORB su ejecución.

El Skeleton es el código servidor generado en base a la definición IDL que está asociado al objeto. Representa el elemento análogo al stub del cliente y se encarga de simular la petición remota del cliente como una petición local a la implementación real del objeto. Contiene las llamadas para realizar el proceso de recuperar del ORB los parámetros con los que se invocó el método, construir la llamada y realizar la petición.

1.2.5.1.3 Tecnologías de Objetos distribuidos

El primer proyecto que intentó desarrollar una tecnología para modelizar sistemas distribuidos complejos con objetos, fue ANSA (1989 – 1991). El proyecto ANSA (Advanced Networked Systems Architecture, Arquitectura de Sistemas Avanzados en Red), surgió como una propuesta a la necesidad de una arquitectura que fuera capaz de abarcar sistemas que soportaran aplicaciones distribuidas y para adoptar un estándar que abarque a toda la industria de la informática.

Actualmente existen distintas tecnologías de desarrollo de sistemas distribuidos basados en objetos:

- Tecnologías Java de Sun Microsystems (Actualmente Oracle Systems):
 - RMI (Remote Invocation Method): Fue el primer framework para crear sistemas distribuidos de Java. Permite, a un objeto que se está ejecutando en una Máquina Virtual Java (VM), llamar a métodos de otro objeto que está en otra VM diferente. Esta tecnología está asociada al lenguaje de programación Java, es decir, que permite la comunicación entre objetos creados en este lenguaje.
 - JINI (Jini Is Not Initials): es una tecnología de Sun Microsystems para el desarrollo de sistemas distribuidos formados por todo tipo de dispositivos que contengan la máquina virtual Java. Actualmente bajo el proyecto Apache River.

- EJB: Enterprise Java Beans es un API que forma parte del estándar de construcción de aplicaciones empresariales J2EE (ahora JEE) de Oracle Corporation. Su especificación detalla cómo los servidores de aplicaciones proveen objetos desde el lado del servidor, que son, precisamente, los EJB. Para más información consulte el tema 60 del temario relativo a arquitecturas JAVA. **"Entorno de desarrollo JAVA para software con distribución"**.
- DCOM (Distributed Component Object Model), de Microsoft: Es un juego de conceptos e interfaces de programa, en el cual los objetos de programa del cliente, pueden solicitar servicios de objetos de programa servidores en otros ordenadores dentro de una red. Esta tecnología está asociada a la plataforma de productos Microsoft.
- CORBA (Common Object Request Broker Architecture), de OMG: Tecnología introducida por el Grupo de Administración de Objetos OMG, creada para establecer una plataforma para la gestión de objetos remotos independiente del lenguaje de programación.

Las tres tecnologías más importantes son RMI, DCOM y CORBA, que serán las que se analicen en los siguientes apartados. EJB es analizado en detalle en el tema 64.

Pueden considerarse finalmente los servicios Web distribuidos como tecnología de última generación para el desarrollo de aplicaciones en red en función de los últimos estándares en servicios Web: XML, SOAP, WSDL y UDDI. Igualmente puede pensarse en los agentes móviles como paradigma de programación alternativo y complementario a los distintos modelos cliente-servidor para diseñar, desarrollar y mantener aplicaciones distribuidas en el ambiente subyacente de las redes de comunicaciones. Estos dos últimos casos, servicios Web y agentes móviles, no son objeto de estudio en este tema ya que se tratan en el tema 55 y 118 del temario.

1.2.5.1.4 RMI (Remote Method Invocation)

RMI permite a los objetos correr en distintas computadoras o en procesos separados para comunicarse con algún otro objeto vía llamadas a métodos remotos. Estas llamadas a métodos remotos suponen tratar un mayor número de excepciones en el sistema que en el caso de llamadas a métodos locales.

El soporte para RMI en Java está basado en las interfaces y clases definidas en los paquetes `java.rmi` y `java.rmi.server`, y para la localización de objetos remotos se emplea el servidor de nombres `java.rmi.Naming`.

En la siguiente figura se muestra en qué consiste la arquitectura Java RMI:

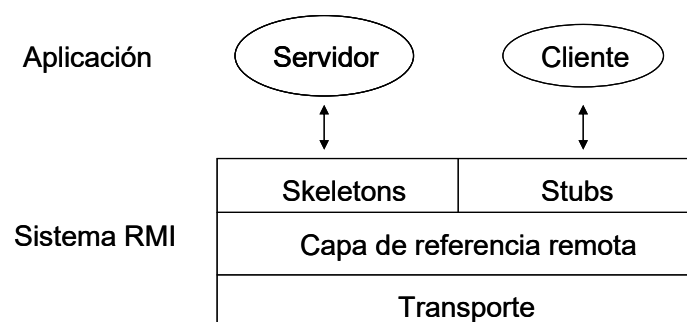


Figura 12.- Arquitectura JAVA RMI

Se cuenta con tres niveles:

- Nivel de transporte, el cual se encarga de las comunicaciones y de establecer las conexiones necesarias.
- Nivel de gestión de referencias remotas, encargado de tratar los aspectos relacionados con el comportamiento esperado de las referencias remotas, como por ejemplo mecanismos de recuperación.

- Nivel stub/skeleton, que se encarga del aplanamiento (serialización) de los parámetros. Cuando un cliente realiza una invocación remota, en realidad hace una invocación de un método del resguardo local. El skeleton recibe las peticiones de los clientes, realiza la invocación del método y devuelve los resultados.

Con RMI se pueden tener distintos niveles de interoperabilidad: nivel bajo, medio o alto. En el nivel alto los programas en diferentes sitios interactúan entre ellos mismos. Estos pueden estar escritos en diferentes lenguajes y correr en diferentes plataformas. Este nivel alto puede conseguirse mediante el uso de CORBA.

1.2.5.1.5 DCOM de Microsoft

DCOM responde a las siglas de Distributed Component Object Model, o lo que es lo mismo Modelo de Objetos de Componentes Distribuidos. Como se ha comentado anteriormente es una tecnología propietaria de Microsoft para desarrollar componentes software distribuidos sobre varios ordenadores y que se comunican entre sí.

Extiende el modelo COM de Microsoft, y la inclusión de la "D" a COM se debe al uso de DCE/RPC (DCE Remote Procedure Call), o sistema de llamada a procedimiento remoto del conjunto de software OSF DCE. DCE responde a las siglas de Distributed Computing Environment, y es un sistema de software para computación distribuida que incluye una serie de servicios que puede utilizarse conjuntamente o de manera separada. Fue creado en 1990 por el consorcio OSF (Open Source Foundation).

El uso de DCE/RPC en COM suponía la posibilidad de resolver algunos de los problemas que presentaba COM, como la recolección de basura distribuida, ya que había que asegurarse de que las referencias mantenidas por clientes de las interfaces eran liberadas cuando no eran necesarias.

DCOM fue uno de los mayores competidores de CORBA, pero actualmente la arquitectura DCOM ha sido abandonada en favor del framework .NET.

1.2.5.1.6 CORBA de OMG

CORBA (Common Object Request Broker Architecture) define la infraestructura para la arquitectura OMA (Object Management Architecture) definida por el OMG (Object Management Group).

El OMG fue fundado en 1989. Formado por un conjunto de organizaciones que cooperan en la definición de estándares para la interoperabilidad en entornos heterogéneos. Actualmente lo componen más de 700 empresas y otros organismos. OMA es una arquitectura de referencia sobre la cual se pueden definir aplicaciones distribuidas sobre un entorno heterogéneo, siendo CORBA la tecnología asociada a esta arquitectura genérica. La OMG desarrolla las especificaciones de CORBA.

Una aplicación definida sobre OMA está compuesta por una serie de objetos distribuidos que cooperan entre sí. Estos objetos se clasifican en tres grupos: servicios, facilidades comunes e interfaces de dominio.

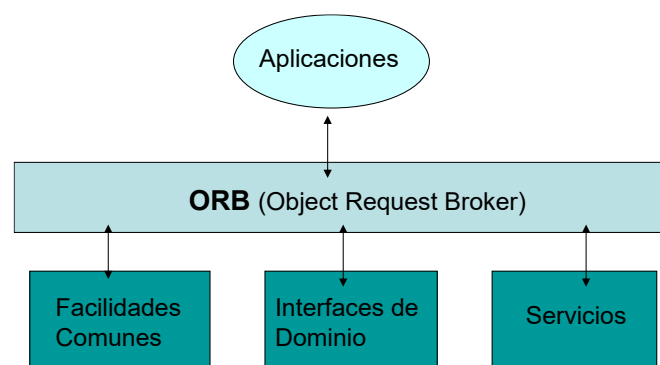


Figura 13 Arquitectura ORB (Object Request Broker)

Los Servicios de OMA proporcionan funciones elementales necesarias para cualquier tipo de entorno distribuido, independientemente del entorno de aplicación. Los tipos de funciones proporcionados son cuestiones tales como la resolución de nombres o la notificación asíncrona de eventos o la creación y migración de objetos. Concede un valor añadido sobre otras tecnologías, como por ejemplo RMI. Las Facilidades Comunes de OMA o Facilidades Horizontales proporcionan funciones comunes a muchas aplicaciones, al igual que los servicios. Están orientadas a usuarios finales (no al desarrollo de aplicaciones). Finalmente las Interfaces de Dominio de OMA o Facilidades Verticales proporcionan funciones complejas restringidas a campos de aplicación muy concretos. Muchos grupos de interés (SIGs) trabajan sobre estas especificaciones.

CORBA ha existido desde 1990, y a partir de 1992 existen implementaciones comerciales, siendo su precursor el citado proyecto ANSA. Es un estándar de objetos distribuidos que permite que una aplicación solicite una operación a ser ejecutada por un objeto distribuido, el cual devolverá los resultados a la aplicación solicitante.

CORBA presenta las siguientes características:

- ✓ Pretende desarrollar aplicaciones distribuidas cuyos componentes puedan colaborar eficiente, confiable, transparente y escalablemente.
- ✓ Proporciona una infraestructura de comunicaciones para la colaboración entre objetos distribuidos heterogéneos.
- ✓ Simplifica la interoperabilidad entre aplicaciones.
- ✓ Pretende aportar a los sistemas distribuidos las ventajas que los lenguajes de programación mediante objetos aportan a la programación no distribuida.
- ✓ Separación entre la interfaz y la implementación. Los clientes dependen de las interfaces, no de las implementaciones.
- ✓ Aporta transparencia de localización y de acceso.

Los componentes de CORBA son:

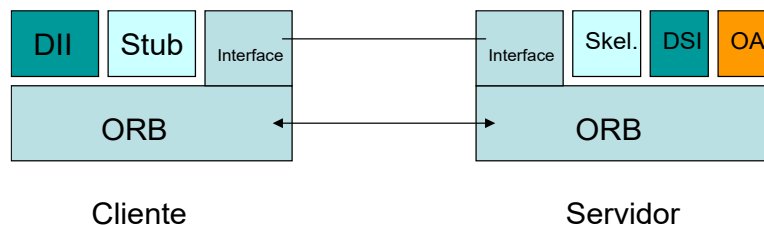


Figura 14 .- Componentes de CORBA

- ✓ Object Request Broker (ORB).
- ✓ Interface Definition Language (IDL).
- ✓ Dynamic Invocation Interface (DII): Es una alternativa al uso de stubs estáticos. Permite que un cliente solicite peticiones a servidores cuyos interfaces se desconocían en fase de compilación.
- ✓ Dynamic Skeleton Interface (DSI): Es una alternativa dinámica al uso de skeletons estáticos definidos en tiempo de compilación del objeto. Es usado por servidores que durante su ejecución pueden arrancar diferentes objetos que pueden ser desconocidos cuando se compiló el servidor.
- ✓ Common Object Services: contruidos sobre CORBA son los servicios que ésta presenta. Se cuenta con servicios de Nombres, de Eventos, de Ciclo de Vida, de Objetos Persistentes, de Transacciones, de Control de Concurrencia, de Relación, de Externalización, de Licencias, de Consulta, de Propiedad, de Tiempo, de Seguridad, de Negociación y de Colección de Objetos.

1.2.5.1.7 Comparativas CORBA, DCOM y RMI

En este apartado se van a comparar los aspectos de la arquitectura CORBA, con aquellos que presentan RMI o DCOM.

Para la comparación entre CORBA y DCOM cabe mencionar el hecho de que CORBA es un estándar abierto y no propietario, como es el caso de DCOM, proporcionando así soporte para diversos sistemas operativos. CORBA es más completo y flexible y da una salida a los legacy systems. Por su parte DCOM está integrado con la tecnología Microsoft y por ello ha tenido una fuerte penetración en el mercado.

En el estudio entre RMI y CORBA, cabe decir que ésta última permite una mayor heterogeneidad en el desarrollo de aplicaciones, ya que RMI sólo se puede desarrollar con Java. CORBA además de las funcionalidades de comunicación, proporciona servicios. Por su parte, RMI puede funcionar sobre CORBA (IIOP) y es mucho más sencillo y cómodo de usar, al igual que permite el paso de objetos por valor y por referencia.

1.2.6 Arquitecturas orientadas a Objetos

En este estilo arquitectónico, los componentes de un sistema encapsulan los datos y las operaciones que deben aplicarse para manipular los datos. La comunicación y coordinación entre los componentes se consigue mediante el paso de mensajes. Los componentes de este estilo son los objetos, o más bien instancias de los tipos de dato abstractos. Los objetos representan una clase de componentes llamados managers, debido a que son responsables de preservar la integridad de su propia representación.

Los componentes del estilo se basan en principios OO: encapsulamiento, herencia y polimorfismo. Son asimismo las unidades de modelado, diseño e implementación, y los objetos y sus interacciones son el centro de las incumbencias en el diseño de la arquitectura y en la estructura de la aplicación.

Las interfaces están separadas de las implementaciones. En general la distribución de objetos es transparente, y no importa si los objetos son locales o remotos. Restricciones: una interfaz pueda ser implementada por múltiples clases. Los componentes y objetos interactúan a través de invocaciones de funciones y procedimientos.

Hay muchas variantes del estilo; algunos sistemas, por ejemplo, admiten que los objetos sean tareas concurrentes; otros permiten que los objetos posean múltiples interfaces.

Para entender los conceptos anteriores que no son desarrollados en otros temas de manera específica se describen a continuación los conceptos principales de Orientación a Objetos a los que también se hará referencia en el tema 89 de técnicas de diseño.

La **Programación Orientada a Objetos** se define como: "un método de implementación en el cuál los programas son organizados como grupos cooperativos de objetos, cada uno de los cuales representa una instancia de alguna clase, y estas clases son miembros de una jerarquía de clases unidas vía relaciones de herencia" [\[Greiff 1994\]](#)

El aspecto más importante al que contribuye la programación orientada a objetos es que la programación deja de ser lineal e imperativa. Se produce un cambio de punto de vista:

"los programas son apéndices de los datos, en lugar de que los datos se usen en programas"

Obviamente, aparece un nuevo concepto: el objeto que es un tipo abstracto de datos con estado (atributos) y comportamiento (operaciones) propios. También aparecen otros conceptos que veremos en profundidad más adelante, como el concepto de jerarquía de tipos, y con él la Herencia y el Polimorfismo.

La orientación a objetos cuenta con las siguientes ventajas:

- ✓ Suministra modelos similares a los del mundo real
- ✓ Facilita el desarrollo de sistemas complejos, generando sistemas más preparados al cambio
- ✓ **Facilita la reutilización de software y del diseño → aumento de la productividad**

- ✓ Permite el desarrollo iterativo de aplicaciones
- ✓ Permite la interoperabilidad entre aplicaciones, aislando las dependencias de las plataformas
- ✓ Los sistemas orientados a objetos son generalmente más pequeños que su equivalente no orientado a objetos, lo que supone menos código y más reutilización
- ✓ Vale para aplicaciones de pequeño y gran tamaño

La orientación a objetos supone así un cambio respecto a los métodos tradicionales, proporcionando un nuevo paradigma para la creación de software. En este nuevo paradigma, los objetos y las clases son los elementos principales, y los métodos, los mensajes y la herencia sirven a estos elementos citados anteriormente.

Estas características intentan resolver el hecho de que cada sistema se construía desde cero, y tiene muy en cuenta los costes de mantenimiento posterior.

De forma general, se puede considerar que un sistema software bajo el paradigma de la Orientación a Objetos está formado por un conjunto de objetos que cooperan para lograr el resultado final, por ejemplo un sistema de dibujo contiene objetos como ventanas, botones, una barra de herramientas, una línea, una imagen

1.2.6.1 Propiedades de la Orientación a Objetos

A continuación se describen las propiedades de la Orientación a objetos, haciendo hincapié en alguna de las características anteriormente identificadas.

Los principios del modelo orientado a objetos son identidad, clasificación, herencia, polimorfismo, abstracción y encapsulación fundamentalmente y en menor grado persistencia, extensibilidad, composición y agregación.

1.2.6.1.1 Identidad

Los datos se cuantifican en entidades discretas y distintas denominadas objetos. Un objeto puede ser cualquier cosa (un fichero, un proceso, etc.) pero tiene su propia identidad inherente, es decir, dos objetos con los mismos valores de sus atributos son dos objetos diferentes. El **handle** o manejador es el identificador único de cada objeto con el que puede ser referenciado unívocamente (su referencia). Dos objetos con los mismos valores de los atributos, pero con identificadores distintos, ocuparan distintas zonas de memoria del sistema. Hay lenguajes que distinguen por tanto entre las operaciones de “=” y de comparación, pudiendo distinguir entre dos objetos iguales (es decir, son el mismo) frente a dos objetos distintos con los mismos valores.

1.2.6.1.2 Clasificación

Los objetos que tienen la misma estructura de datos (atributos) y el mismo comportamiento (operaciones), están agrupados en la misma clase. Una clase es una abstracción que describe las propiedades esenciales de los objetos o entidades particulares al dominio de un problema concreto. Un objeto es la instancia de la clase (objeto $\leftrightarrow$ instancia).

1.2.6.1.3 Herencia

La Herencia es el mecanismo mediante el cual una clase (subclase) adquiere las propiedades de otra clase jerárquicamente superior (superclase, clase base). La herencia proporciona el mecanismo para compartir automáticamente métodos y datos entre clases, subclases y objetos.

En la figura siguiente se tienen dos clases Taxi y Autobús.

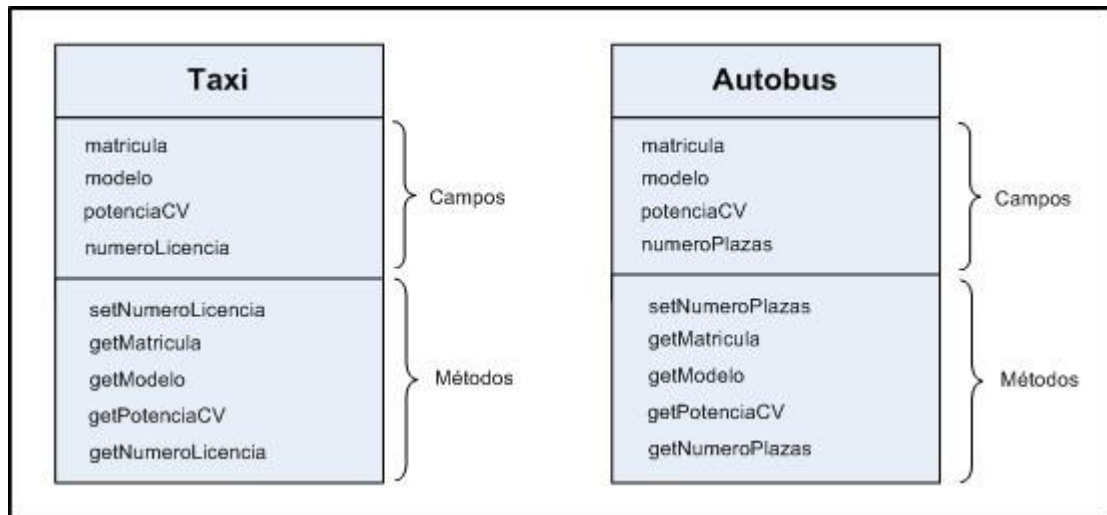


Figura 15.- Clases Taxi y Autobús que comparten atributos y métodos

Si nos fijamos encontramos lo siguiente:

- La definición de clases nos permite identificar campos y métodos que son comunes a Taxis y Autobuses. Si implementamos ambas clases tal y como se muestra en la figura anterior, incurriremos en duplicidad de código. Por ejemplo si el campo matricula es en ambas clases un tipo String, el código para gestionar este campo será idéntico en ambas clases.
- La definición de clases nos permite identificar campos y métodos que difieren entre una clase y otra. Por ejemplo en la clase Taxi se gestiona información sobre un campo denominado numeroDeLicencia que no existe en la clase Autobus.
- Conceptualmente podemos imaginar una abstracción que engloba a Taxis y Autobuses: ambos podríamos englobarlos bajo la denominación de "Vehículos". Un **Taxi** sería un tipo de Vehículo y un **Autobus** otro tipo de Vehículo.
- Si la empresa añade otros vehículos como minibuses, tranvías, etc. manteniendo la definición de clases "individualizada" se duplicaría código. Por ejemplo, un minibus también tendría matrícula, potencia... y los métodos asociados.

La duplicidad de código nos implicará problemas de mantenimiento, por ejemplo si tenemos simplemente que modificar el código de un método que aparece en distintas clases. El tener el código duplicado nos obliga a tener que hacer dos o más modificaciones en sitios distintos. Pueden ser dos modificaciones, tres, cuatro o n modificaciones dependiendo del número de clases que se vieran afectadas, y esto a la larga genera errores al no ser el mantenimiento razonable.

La herencia nos permite definir una clase como extensión de otra. De esta manera decimos "la clase "hija" o "subclase" tiene todas las características de la clase "padre"/"superclase" y además sus características particulares". Todo lo que es común a ambas clases (Taxi y Autobus) queda comprendido en la clase "superior", mientras lo que es específico, queda restringido a las clases "inferiores". En nuestro ejemplo se define la clase **Vehículo**, de forma que la clase Taxi tuviera todas las propiedades de la clase Vehículo, más algunas propiedades y métodos específicos. Lo mismo ocurriría con la clase Autobus y otras que pudieran "heredar" de Vehículo, tal y como aparece en la siguiente figura.

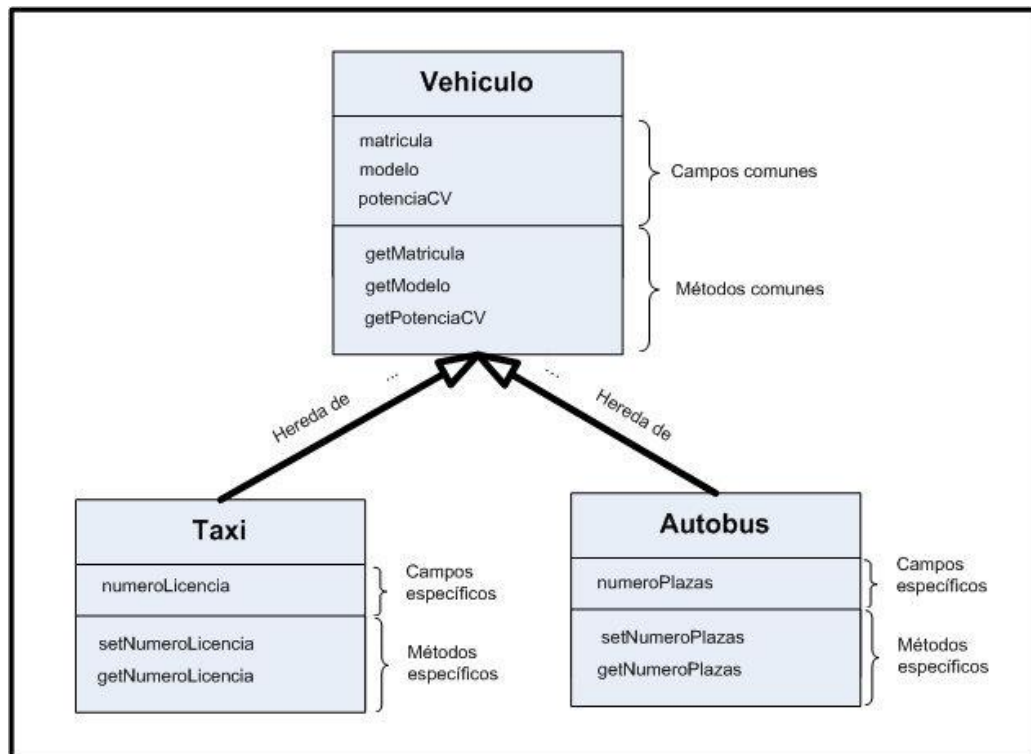


Figura 16 .- Clases Taxi y Autobús que heredan atributos y métodos de la clase vehículo

La herencia proporciona por tanto relaciones entre clases del tipo “es-un” (is-a), que ya se vieron en el Modelo Entidad Relación Extendido.

1.2.6.1.4 Tipos de Herencia

La herencia puede ser de distintos tipos:

- ✓ **Implementación:** implica que la clase hija hereda la implementación de métodos de la clase padre.
- ✓ **Interfaz:** implica que la clase hija hereda el interfaz (pero no la implementación de las operaciones).
- ✓ **De clase:** combina la herencia de implementación y de interfaz.

Igualmente se puede clasificar en simple o múltiple, según que una subclase herede los datos y métodos de una sola clase o de más de una. Como se ha comentado anteriormente no todos los lenguajes soportan herencia múltiple, por los posibles problemas de ambigüedad. Estos problemas deben ser resueltos por el entorno y se refiere a ellos de manera genérica como ligadura dinámica.

Ligadura o enlazado

La ligadura se encarga de ligar o relacionar la llamada a un método con el cuerpo del método que se ejecuta finalmente.

Existen dos tipos de ligadura: estática y dinámica.

Ligadura estática:

Consiste en realizar el proceso de ligadura en tiempo de compilación según el tipo declarado del objeto al que se manda el mensaje. La utilizan (**java**) los métodos de clase y

los métodos de instancia que son privados o final (ya que estos últimos no pueden ser sobrescritos).

Ligadura dinámica:

Consiste en realizar el proceso de ligadura en tiempo de ejecución (ya que no puede resolverse en tiempo de compilación) siendo la forma dinámica del objeto la que determina la versión del método a ejecutar.

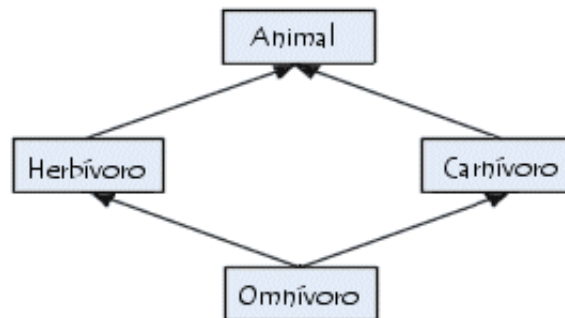


Figura 17 .- Ejemplo de clases con herencia múltiple

Imaginemos que se quiere realizar un proceso de herencia múltiple. Debe estar definido el orden de precedencia, y como resolver los problemas cuando un objeto de la clase omnívoro herede un atributo o método de las clases padre.

Funcionamiento de la ligadura dinámica

Cuando existe un conflicto entre un método de una superclase y un método de la subclase, el comportamiento correcto es que el método de la subclase sobrescriba al de la superclase.

Si estamos llamando a un método de la subclase desde una variable que ha sido declarada del tipo de la superclase. ¿Cómo se consigue que funcione correctamente?

Significa que la forma dinámica del objeto determina la versión de la operación que se aplicará.

Esta capacidad de las operaciones para adaptarse automáticamente a los objetos a los cuales se aplican es una de las propiedades más importantes de la orientación a objetos.

1.2.6.2 Polimorfismo

El Polimorfismo es la propiedad por la cual un mismo mensaje puede originar conductas diferentes al ser recibido por objetos diferentes. El polimorfismo es consecuencia de la herencia. Las funciones miembro de una clase base pueden ser sustituidas en una clase derivada mediante la duplicación de su declaración en la clase hija. Por lo tanto, los objetos de las dos clases pueden reaccionar ambos a los mismos mensajes pero lo harán de diferentes maneras.

También se puede dar el polimorfismo variando los parámetros que definen un método. Es decir una clase puede tener dos o más métodos que se llaman igual, como por ejemplo ocurre en el caso de los constructores, y tendrían o podrían tener comportamientos distintos. Este tipo de polimorfismo se denomina "Sobrecarga".

Las diferencias fundamentales entre la sobrecarga y el polimorfismo son:

- ✓ La sobrecarga es estática y se resuelve en tiempo de compilación, y el polimorfismo en general es dinámico y se resuelve en tiempo de ejecución.
- ✓ La sobrecarga es más eficiente (veloz) en tiempo de ejecución.

- ✓ La sobrecarga afecta a una única clase, y el “polimorfismo” a varias clases, normalmente relacionadas por una relación de herencia, aunque no en todas las situaciones es necesaria la herencia.

Profundizando en el tema, los tipos de polimorfismo se clasifican de la siguiente manera:

- ✓ Polimorfismo estático o ad-hoc (sobrecarga): es aquél en el que los tipos a los que se aplica el polimorfismo deben ser explícitos y declarados uno por uno antes de poder ser utilizados
- ✓ Polimorfismo dinámico (o polimorfismo paramétrico): es aquél en el que el código no incluye ningún tipo de especificación sobre el tipo de datos sobre el que se trabaja. Así, puede ser utilizado a todo tipo de datos compatible.

A su vez el polimorfismo dinámico se divide en:

- ✚ Paramétrico: Permite escribir una función o tipo de datos de manera genérica, de manera que puede manejar valores uniformemente sin depender del tipo. Este tipo un lenguaje más expresivo a la vez que mantiene un fuerte tipado estático.
- ✚ Subtipado: (o de inclusión) en el que permite a una función o método aceptar tipos que sean subtipo del tipo definido originalmente.
- ✚ Multitipo o politipo: También conocido como de genericidad de tipo de datos. El más amplio que “polimorfismo” a secas y está relacionado con el concepto de programación genérica y Lenguajes funcionales.

En el caso de los operadores (+, *, =, ...), en un lenguaje orientado a objetos, éstos son polimórficos porque hacen la operación dada en función de los objetos que reciben el mensaje de operación.

Se puede ampliar información en la bibliografía recomendada.

1.2.6.3 Encapsulación y Abstracción

Encapsulación cualidad por la que los datos de un objeto sólo pueden ser manipulados mediante los mensajes y métodos predefinidos. Es decir, los datos relativos a algún objeto están almacenados junto con el proceso que crea y manipula esos datos, tal y como se comentó anteriormente al hablar de los Tipos Abstractos de Datos (TAD).

La encapsulación es un mecanismo para conseguir la abstracción de datos.

1.2.6.4 Persistencia

Un objeto en software ocupa un determinado espacio de memoria y existe durante una cierta cantidad de tiempo. La persistencia es la cualidad que se refiere a la permanencia del objeto; es decir, al tiempo durante el cual se le asigna espacio y puede ser accesible (EJB EntityBeans). Uno de los problemas más habituales en programación es la gestión de la memoria y evitar que se produzcan “memory Leaks”, o pérdida paulatina de memoria hasta que el equipo se queda sin recursos. La recuperación automática del espacio de memoria se denomina normalmente recolección de basura, “Garbage Collection” y es implementada por lenguajes modernos como JAVA.

1.2.6.5 Extensibilidad

Es la capacidad de un programa para ser fácilmente alterado de forma que pueda tratar con nuevas clases de entrada. Mediante esta propiedad, los objetos pueden ser usados para almacenar y procesar muchos tipos diferentes de datos (gráficos, texto, vídeo, sonido) simplemente añadiendo clases que traten los tipos de datos que sean necesarios.

En la Programación Orientada a Objetos, la herencia, el polimorfismo y la composición y agregación son mecanismo de extensibilidad del lenguaje, frente al mero “incremento de procedimientos o funciones” de la programación estructurada.

1.2.6.6 Reusabilidad

Es la capacidad de producir componentes reutilizables para otros diseños o aplicaciones. En la OO se consigue de una forma natural mediante el diseño de los componentes para residir en algún repositorio estándar y la utilización de algún repositorio coordinador y herramientas CASE integradas orientadas a objetos (por ejemplo, usando el framework .NET o Java).

1.2.6.7 Composición/Agregación

Representa el principio de la partición. Una clase B es parte de una clase A. La agregación proporciona relaciones del tipo “parte-de” o “formado por”. Como se puede ver en el siguiente ejemplo, un gráfico está formado por múltiples figuras.

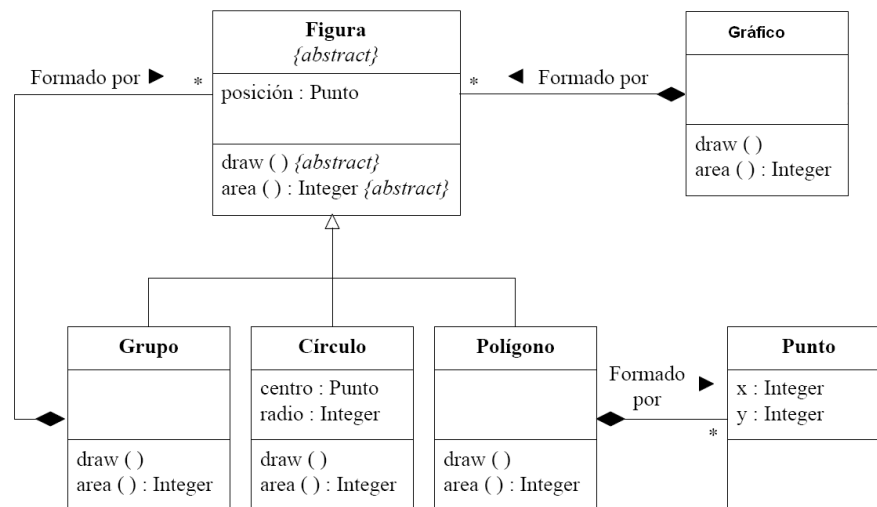


Figura 18 .- Relaciones de composición entre clases

En este caso, el código de los métodos del agregado (Gráfico) delega parte de su implementación en métodos de sus partes (Figura). Por ejemplo, el *area()* del Gráfico estará dada por la función de área que corresponda de las Figuras. Es decir, la clase Gráfico “hereda” su comportamiento sin estar en una relación de herencia “es-un”. Por lo tanto, la reutilización de código puede hacerse mediante la herencia o mediante la composición.

Dentro de los tipos de asociaciones entre clases (ver el tema 84 del temario en relación a las asociaciones de clases) la agregación es más débil que la composición. En el caso de la composición, las clases que forman parte (componen) de otra clase principal, no tienen sentido si no existe la clase que las “compone”. Por ejemplo el sistema eléctrico de un coche, el motor de un coche, el parabrisas etc. En el caso de la agregación, las clases agregadas pueden tener sentido independientemente de la clase que las agrega. Por ejemplo la clase **Zoologico**, agregaría a la clase **pantera**, **leon**, **hiena**, etc. Si no existiera la clase zoológico, las clases de los animales podrían existir de manera independiente, o formar parte (agregación) de otra clase, como por ejemplo la clase **AnimalesJungla**.

La distinción entre composición y agregación es conceptual, desde el punto de vista del código son indistinguibles.

1.2.7 Arquitectura MVC

En este apartado se planteará una visión general de la arquitectura MVC que ha sido muy utilizada durante mucho tiempo en ingeniería de software. Casi todos los lenguajes de programación soportan MVC con ligeras variaciones, pero conceptualmente sigue siendo el mismo concepto.

MVC separa la aplicación en tres componentes: Modelo, Vista y Controlador (Model, View y Controller en Inglés).

MVC se considera un patrón de arquitectura de software que separa los datos y la lógica de negocio de una aplicación de la interfaz de usuario y el módulo encargado de gestionar los eventos y las comunicaciones. Este patrón plantea la separación del problema en tres capas: la capa **modelo**, que representa la realidad; la capa **controlador**, que conoce los métodos y atributos del modelo, recibe y realiza lo que el usuario quiere hacer; y la capa **vista**, que muestra un aspecto del modelo y es utilizada por la capa anterior para interactuar con el usuario.

- ✓ **Modelo:** El modelo representa la forma de los datos y la lógica de negocio. Mantiene los datos de la aplicación. Los objetos de modelo recuperan y almacenan el estado del modelo en una base de datos. El modelo se refiere a la lógica de datos y de negocio.
- ✓ **Vista:** La Vista es una interfaz de usuario. La vista permite la visualización de los datos utilizando el modelo para el usuario y también les permite modificar los datos.
- ✓ **Controlador:** El controlador controla la solicitud del usuario. Típicamente, el usuario interactúa con la vista, que internamente remite la solicitud de URL adecuada, esta solicitud será manejada por un controlador. El controlador genera la vista apropiada con los datos del modelo como respuesta. El controlador es un controlador de peticiones.

MVC se basa en las ideas de reutilización de código y la separación de conceptos, características que buscan facilitar la tarea de desarrollo de aplicaciones y su posterior mantenimiento.

La siguiente figura ilustra la interacción entre Modelo, Vista y Controlador.

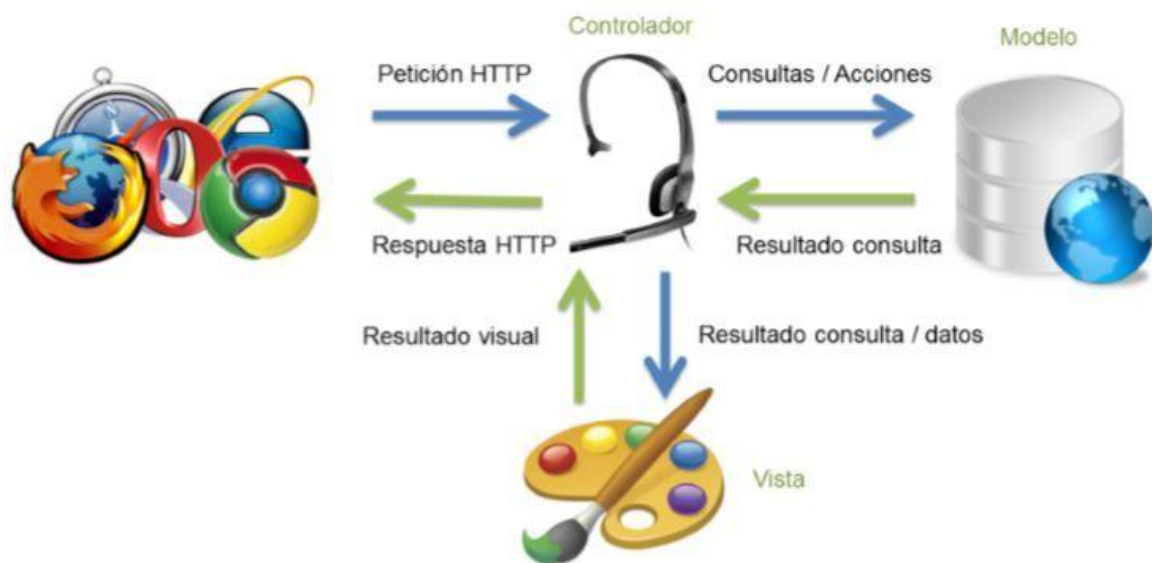


Figura 19 .- Arquitectura MVC - Modelo, Vista y Controlador

Según la figura anterior, cuando el usuario introduce una URL en el navegador, va al servidor, este llama al controlador apropiado. A continuación, el controlador utiliza la vista y el modelo adecuados, crea la respuesta y la devuelve al usuario.

Aunque originalmente MVC fue desarrollado para aplicaciones de escritorio, ha sido ampliamente adaptado como arquitectura para diseñar e implementar aplicaciones web en los principales lenguajes de programación. Se han desarrollado multitud de frameworks, comerciales y no comerciales, que implementan este patrón en distintas tecnologías, .NET, Java, PHP, etc. Una lista bastante amplia puede consultarse en internet en las referencias bibliográficas. Estos frameworks se

diferencian básicamente en la interpretación de cómo las funciones MVC se dividen entre cliente y servidor.

Los primeros frameworks MVC para desarrollo web planteaban un enfoque de cliente ligero en el que casi todas las funciones, tanto de la vista, el modelo y el controlador recaían en el servidor. En este enfoque, el cliente manda la petición de cualquier hipervínculo o formulario al controlador y después recibe de la vista una página completa y actualizada (u otro documento); tanto el modelo como el controlador (y buena parte de la vista) están completamente alojados en el servidor. Como las tecnologías web han madurado, ahora existen frameworks como JavaScriptMVC, Backbone, AngularJS o jQuery que permiten que ciertos componentes MVC se ejecuten parcial o totalmente en el cliente.

1.2.8 Arquitecturas en capas

En este estilo arquitectónico, hay varias capas definidas, cada una de ellas realiza operaciones que se acercan progresivamente al conjunto de instrucciones de la máquina. En la capa externa los componentes sirven a las operaciones de interfaz del usuario. En la capa interna los componentes sirven como interfaz con el sistema operativo. Las capas intermedias proporcionan servicios de utilidades y de SW de aplicaciones. Esta división puede realizarse en un único equipo o de manera distribuida en varios equipos.

Este estilo es ampliamente conocido, siendo de amplia difusión y el más ampliamente utilizado en aplicaciones de propósito general con el desarrollo de la Web e Internet, donde la división en capas se aplica de manera natural. Se desarrolla en profundidad en el tema 55 y en el tema 89.

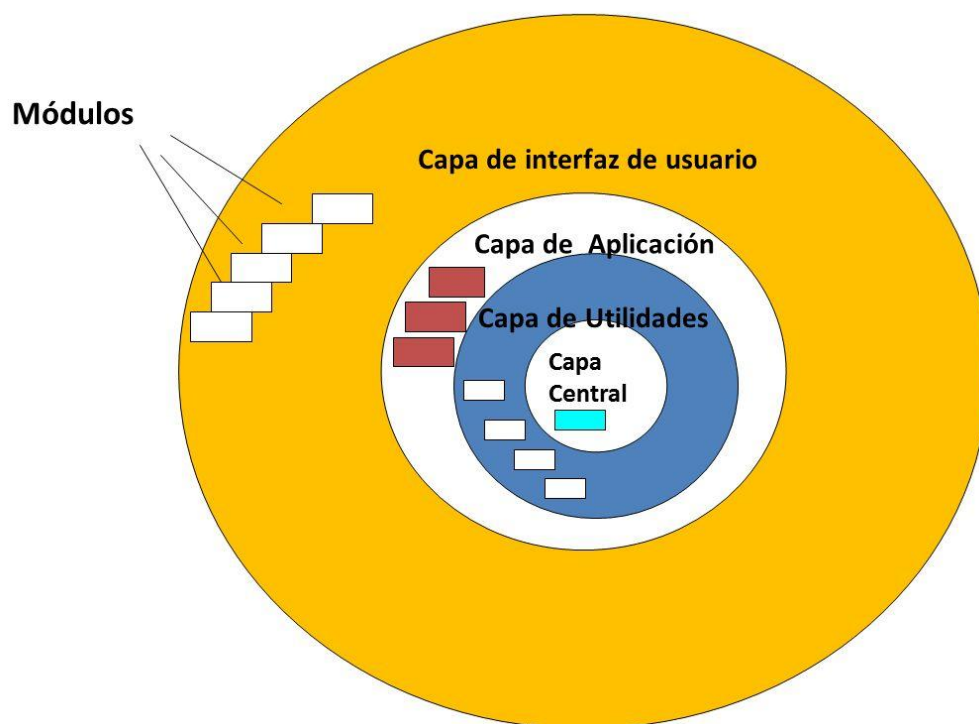


Figura 20 .- Arquitecturas en capas

Las ventajas de la arquitectura en capas son:

- ✓ Facilitar la descomposición del problema en varios niveles de abstracción.
- ✓ Soportar fácilmente la evolución del sistema, los cambios sólo afectan a las capas vecinas.
- ✓ Se pueden cambiar las implementaciones respetando las interfaces con las capas adyacentes.

Como desventajas principales están el que no todos los sistemas pueden estructurarse en capas y a menudo es difícil encontrar la separación en capas adecuadas.

1.2.9 Arquitecturas de sistemas de Tiempo Real

Se definen los sistemas de tiempo real como aquellos que reciben información del entorno donde se encuentran, comprobando su estado y actuando sobre él para restituir la normalidad si se produjeran desviaciones significativas en su comportamiento o situación.

Los sistemas de tiempo real se suelen emplear, fundamentalmente, en aplicaciones dedicadas a construir sistemas de control, por ejemplo, control de seguimiento de satélites, control de procesos industriales, etc. Estos sistemas no se deben confundir con aquellos otros, como, por ejemplo, los de reserva de billetes, cajeros automáticos, etc., que, si bien necesitan tener la información actualizada según se producen los cambios, no son críticos en el tiempo, ya que deben responder a entradas externas originadas por el ser humano y, por tanto, sólo requieren tiempos de respuesta del orden de segundos. Es decir, no se deben confundir los sistemas de tiempo real con los sistemas que funcionan “en tiempo real”, y cuya principal característica es la alta interactividad y el alto régimen transaccional.

Por lo tanto, se puede decir que un sistema es de tiempo real si su tiempo de respuesta permite afectar (controlar y regular) al medio en el cual opera.

Los principales objetivos que ha de alcanzar un sistema de tiempo real son:

- ✓ Satisfacer las fuertes restricciones en el tiempo de respuesta (en torno a milisegundos), por lo que los tiempos de los componentes que intervienen en el sistema serán críticos.
- ✓ Información permanentemente actualizada.
- ✓ Fuertes restricciones de funcionamiento en la ejecución de los procesos.
- ✓ El sistema deberá permanecer prácticamente inactivo para poder atender con la mayor rapidez posible cualquier evento de entrada y poder reaccionar con la mayor celeridad posible.
- ✓ Alto rendimiento del sistema, tanto del hardware como del software, funcionando conjuntamente, lo que implica fuertes problemas de integración de diversas tecnologías.

Los objetivos citados son especialmente críticos en la actuación de un sistema de tiempo real, y para poder alcanzarlos es imprescindible que el ordenador en el que se va a implantar el sistema disponga de un sistema operativo de tiempo real.

Los sistemas operativos de tiempo real se caracterizan por disponer de las siguientes funciones:

- ✓ Manejo eficaz de interrupciones.
- ✓ Manejo sencillo, pero potente, de prioridades, permitiendo que puedan modificarse dinámicamente incluso durante la ejecución de los procesos.
- ✓ Gestión de memoria real, no virtual.
- ✓ Mecanismos de sincronización (semáforos o señales) para asegurar la exclusión mutua de los procesos sobre los datos y recursos compartidos.
- ✓ Mecanismos potentes y versátiles de comunicación entre procesos.
- ✓ Bloqueo de los procesos cuando están en espera y, normalmente, gestión FIFO de las colas de procesos bloqueados.

Los sistemas de tiempo real están constituidos, normalmente, por varios procesos o tareas concurrentes, cada una de ellas secuencial, produciéndose la concurrencia al ejecutarse varias tareas asíncronas a diferentes velocidades. Por ello, el acceso a los distintos recursos del sistema compartidos por cada proceso se realizará en instantes imprevisibles, lo que conlleva que puedan presentarse problemas difíciles de prever durante el diseño y de depurar en el caso de que se presenten durante la ejecución. Con el fin de evitar estos problemas, cada tarea necesitará, de vez en cuando, comunicarse y sincronizarse con las otras. El elemento encargado de resolver estos problemas de comunicación entre procesos es el sistema operativo.

Los problemas de comunicación entre procesos se pueden jerarquizar en tres áreas: la exclusión mutua, el sincronismo y el bloqueo o “dead lock”.

1. Exclusión mutua. Los recursos de un sistema pueden ser compartibles por varios procesos, por ejemplo, la CPU o un fichero de sólo lectura, y no compartibles, ya sea por su propia naturaleza física, tal como una impresora, etc., o por consideraciones lógicas, como, por ejemplo, un fichero en el que se va a escribir una información para actualizarlo. Pues bien, el sistema operativo debe garantizar que se cumple el principio de exclusión mutua, esto es, que un recurso no compartible sólo puede ser accedido por un proceso a la vez.
2. Sincronismo. Dado que los procesos se ejecutan de una manera asíncrona y con una velocidad imprevisible, es necesario que el sistema operativo garantice que en determinados puntos de la ejecución se produzca una sincronización entre los procesos.
3. Bloqueo. El sistema operativo deberá resolver los problemas de bloqueo, congestión o abrazo mortal (“dead lock”), los cuales se producen en aquellas situaciones en las que un proceso no arranca porque necesita disponer de un recurso no compartible que está asignado a otro proceso, y éste no libera dicho recurso porque lo necesita para su ejecución.

Todos estos problemas los resuelve el sistema operativo a través de un mecanismo denominado semáforo, que se define como “una variable entera no negativa y compartida por varios procesos”.

Al semáforo se le dota de dos operaciones distintas e indivisibles, y cuando empieza una operación sobre el mismo, el sistema operativo no puede permitir que se interrumpa hasta que termine. Estas dos operaciones son:

- ✓ Operación SIGNAL: Aumenta en una unidad el valor inicial que tuviera el semáforo sobre el que se aplica. Si este valor inicial fuera cero, además de poner valor uno al semáforo, pone en marcha otro proceso, si lo había detenido.
- ✓ Operación WAIT: Disminuye en una unidad el valor inicial que tuviera el semáforo sobre el que se aplica. Si este valor inicial fuera cero, como el semáforo, por su propia definición, no puede tomar valores negativos, lo que hace esta operación es detener el proceso.

Un sistema en tiempo real debe resolver satisfactoriamente los siguientes dos problemas:

- ✓ El problema de la cena de los filósofos: es un problema clásico para representar el problema de la sincronización de procesos en un sistema operativo. 5 filósofos sentados a la mesa, un palillo por filósofo, necesitan 2 palillos para comer. DeadLock (Edsger Dijkstra en 1965).
- ✓ El problema del barbero durmiente es un problema de sincronización. En una barbería en la que trabaja un barbero que tiene un único sillón de barbero y varias sillas para esperar. Cuando no hay clientes, el barbero se duerme. Un nuevo cliente, o despierta al barbero o —si el barbero está afeitando a otro cliente— se sienta en una silla (o se va si todas las sillas ocupadas). El Objetivo consiste en realizar la actividad del barbero sin que ocurran condiciones de carrera.

La solución implica el uso de semáforos y objetos de exclusión mutua para proteger la sección crítica.

1.2.9.1 Análisis de sistemas de tiempo real.

En un sistema de tiempo real, dado un conjunto de procesos, éstos se deben ejecutar en un cierto orden y bajo unas fuertes restricciones de tiempo y de hardware. Quiere esto decir que el problema de los sistemas de tiempo real es básicamente de planificación de recursos y, por tanto, el método elegido deberá planificar éstos y los procesos de una forma óptima.

Fundamentalmente, la estrategia para resolver un problema de tiempo real se puede descomponer en los siguientes pasos:

1. Formulación del problema: Definiendo claramente los objetivos, la interdependencia de los distintos parámetros del sistema, la disponibilidad de los recursos y un índice de rendimiento para poder evaluar las distintas soluciones que se propongan.
2. Selección de la técnica: Es decir, de los medios que permitan obtener la solución deseada según la formulación del problema. Dicha solución debe ser óptima, o como mínimo buena.
3. Solución: La técnica y métodos apropiados se deben aplicar para alcanzar la solución en presencia de las restricciones del sistema.

Los sistemas de tiempo real se basan en la retroalimentación de la información siendo, por tanto, sistemas de control del entorno. La base para el análisis de un sistema de tiempo real pasa por la teoría de los sistemas lineales, la cual estudia las interacciones y comportamiento de cada componente, pudiéndose representar dichos sistemas mediante modelos matemáticos, diagramas de bloque y grafos de flujo de señales.

- ✓ Los modelos matemáticos se basan en sistemas de ecuaciones que describen detalladamente las relaciones entre las distintas variables.
- ✓ Los diagramas de bloques también tratan de describir y analizar sistemas lineales, es decir, sistemas con relaciones causa-efecto entre sus componentes.
- ✓ Los grafos de flujo de señales, cuyo principal exponente son las Redes de Petri, permiten describir la concurrencia y son la base para la construcción de diversas herramientas para el estudio de estos problemas.

Las Redes de Petri se basan en elementos gráficos formados por Lugares (place), Transiciones (transition) y Testigos (token), de forma que los lugares se unen a las transiciones. En los lugares se colocan los testigos y cuando todos los lugares conectados a una transición tienen testigo, la transición está “cargada” y en ese momento se “dispara” pasando los testigos desde los lugares precedentes hasta los que cuelgan de la transición.

1.2.9.2 Diseño de sistemas de tiempo real

En general, los métodos de diseño, o bien presentan limitaciones para los sistemas de tiempo real, o bien son muy complicados de llevar a la práctica. No obstante, el Diseño Estructurado satisface prácticamente las necesidades de estos sistemas y es fácil de aprender y de llevar a la práctica, por lo que con pocas modificaciones se puede adaptar a los sistemas de tiempo real. Complementario a la Técnica de DFD está la de Diagrama de Flujos de Control, ampliación propuesta por Ward y Mellor para sistemas de tiempo real que permite representar procesos de control, flujos de control, almacén de control y flujos continuos de información en el tiempo.

Un método adecuado para el diseño de sistemas en tiempo real debe tener los siguientes requisitos:

1. Requisitos del software de tiempo real: Un software diseñado para un sistema de tiempo real debe perseguir los siguientes objetivos :
 - ✓ Consistencia, es decir, obtener una respuesta consistente que satisfaga los requisitos de tiempo.
 - ✓ Funcionalidad, es decir, alcanzar los requisitos funcionales especificados.
 - ✓ Comportamiento razonablemente rápido.

Para alcanzar estos objetivos, el software de tiempo real ha de ser:

- ✓ Predecible de acuerdo con las restricciones de tiempo necesarias y evitar bucles difícilmente evaluables.
- ✓ Fiable, esto es, basado en lenguajes de programación que permitan programar de forma robusta y segura.
- ✓ Sincronización y división de tareas, o sea, que se pueda dividir el sistema en subsistemas y

- éstos en tareas que se puedan ejecutar concurrentemente, y por tanto, contar con los suficientes mecanismos de sincronización entre las mismas.
 - ✓ Modular: Que el diseño de las tareas y procesos se pueda dividir en módulos.
 - ✓ Interfaz: El lenguaje debe permitir el acceso al hardware y el manejo de interrupciones de forma flexible y segura.
 - ✓ Facilidad de mantenimiento, ya que la esperanza de vida de estos sistemas es muy larga y el coste de mantenimiento es muy alto, por tanto los programas deben ser fáciles de mantener, es decir, de leer, entender y modificar.
2. Diseño orientado al flujo de datos: Es muy adecuado para el diseño de sistemas en tiempo real ya que los datos en este tipo de sistemas se pueden considerar como un flujo que entra al sistema y que se transforma mediante tareas software para obtener unos datos de salida.
 3. Comunicación y sincronización entre tareas: Para que las tareas que componen un sistema de tiempo real realicen una determinada operación deben comunicarse datos entre sí y sincronizarse para que no surjan problemas.
 4. Ocultación de la información: Las decisiones claves de diseño deben poder conocerse sólo en un módulo determinado, reduciendo al mínimo la compartición de información entre módulos.

1.2.10 Sistemas Transaccionales

Una transacción consiste en un conjunto de eventos que deben ser llevados a cabo como una unidad indivisible de trabajo, en la que todos y cada uno de ellos tienen éxito o todos y cada uno de ellos son rechazados.

Un suceso externo que involucra el traslado de algo de valor entre dos o más entidades.

En términos de procesamiento, las transacciones se confirman o se anulan.

- ✓ Para que una transacción se confirme, todos los participantes deben garantizar la permanencia de los cambios efectuados en los datos.
- ✓ Los cambios deben conservarse aunque el sistema se bloquee o tengan lugar otros eventos imprevistos. Basta con que un solo participante no pueda garantizar este punto para que la transacción falle en su totalidad.
- ✓ Todos los cambios efectuados en datos dentro del ámbito de la transacción se deshacen hasta un punto específico establecido.
- ✓ Las transacciones pueden ser externas, como el pago de una factura a un proveedor, o internas, como el envío de material desde el almacén al área de fabricación.

Como esta definición se ajusta a un gran número de actividades cotidianas, se ha convertido en una verdadera filosofía de diseño aplicable a innumerables áreas, especialmente en el mundo de los negocios y eventos financieros que implican transferencias de dinero.

Por este motivo, las transacciones tienen que realizarse rápidamente y con mínimos riesgos.

Un ejemplo típico. Es el de la transferencia de fondos entre dos cuentas corrientes de un banco. Si queremos transferir, pongamos 5000€ de la cuenta corriente de Armando a la de Benito (A y B a partir de ahora) y las cuentas tienen, respectivamente, 20000€ y 0€ de saldo los pasos lógicos serían:

1. Comprobar si en la cuenta A hay dinero suficiente.
2. Restar 5000€ de la cuenta de A, con lo que su saldo pasa a ser de 15000€
3. Sumar 5000€ a la cuenta de B, con lo que los saldos quedan A=15000€ y B=5000€

Ahora bien, si entre el paso 2 y el 3 el sistema sufre una parada o error inesperado las cuentas quedarían como A=15000 y B=0 con lo cual... Se han volatilizado 5000€ y presumiblemente ni Armando ni Benito estarán contentos, y hubiesen preferido que la transacción nunca hubiese sido iniciada.

Este ejemplo ilustra porqué las transacciones tienen un comportamiento deseado de todo o nada, o se realiza completamente o no debe tener ningún efecto.

Las transacciones pueden ser:

- ✓ Recíprocas. Intercambios en los que cada participante recibe y sacrifica un valor. Por ejemplo, adquisiciones o ventas de bienes o servicios.
- ✓ No recíprocas. Transacciones en las que una entidad incurre en un pasivo o transfiere un activo a otra entidad (o recibe un activo o a la cancelación de un pasivo), sin recibir (o entregar) directamente un valor a cambio del otro.

Para la implementación de Sistemas Transaccionales se suele utilizar un monitor de transacciones, que engloba un conjunto de uno o más componentes (no necesariamente en el sentido de componentes de objetos) que brindan el soporte para el diseño, desarrollo, configuración y operación de aplicaciones “confiables” de transacciones distribuidas. Esto significa que garantizan las propiedades ACID para las aplicaciones, y también incluye la puesta en marcha de los procesos servidores, la canalización de los mensajes de solicitud/respuesta y algún tipo de supervisión y equilibrio de cargas.

Un monitor de transacciones puede ser concebido como una estructura preconstruida (built-in block) que ayuda a desarrollar y administrar una aplicación cliente/servidor, garantizando las propiedades ACID y un alto rendimiento.

Los sistemas transaccionales son sistemas basados en el flujo de datos. El modelo fundamental responde al del flujo de información desde una fuente de datos, realización de ciertas transformaciones y depósito o entrega de la información en un “sumidero” o destino del flujo. La herramienta gráfica que se utiliza para representar este modelo es el Diagrama de Estructura de Cuadros de Constantine para Diagramas de Flujo de Datos con características de transacción que se desarrolla en el tema 89.

1.2.10.1 Características de las transacciones (ACID)

Una transacción tiene que poseer las siguientes características, englobadas generalmente dentro del acrónimo ACID (Atomicity, Consistency, Isolation, Durability):

1. Atomicidad: cualquier cambio de estado que produce una transacción es atómico. A pesar de que una transacción está compuesta por un número cualquiera de eventos, el sistema las debe considerar como una única operación, la cual puede tener éxito; en tal caso se hacen permanentes los cambios generados por cada evento componente de la transacción; o fracaso, en este caso el sistema queda en el mismo estado, como si la transacción nunca hubiera ocurrido.
2. Consistencia: propiedad que asegura que una transacción no romperá con la integridad de una base de datos, pues respeta todas las reglas y directrices de ésta. Todos los cambios provocados por la transacción deben dejar al sistema en un estado correcto. El sistema es llevado desde un estado válido a otro estado válido, producto de la acción de una transacción.
3. Aislamiento: propiedad que asegura que no se afectarán entre sí las transacciones. En otras palabras, dos o más transacciones sobre los mismos datos no generarán un problema.
4. Durabilidad: propiedad que asegura la persistencia de una transacción, es decir, una vez que la transacción quedó aceptada no podrá deshacerse aunque falle el sistema.

La filosofía de crear sistemas cliente/servidor bajo el concepto de transacciones con propiedades ACID brinda a los desarrolladores una gran simplicidad, y actualmente están presente en la casi totalidad de las aplicaciones cliente/servidor de la actual generación de sistemas transaccionales.

Su principal espectro de aplicación resulta ser en actividades de naturaleza breves, históricamente

el desarrollo de transacciones se debió a su utilidad para aplicaciones bancarias,

Resultan inadecuadas para:

- ✓ Manejo de transacciones de negocios que se extienden periodos prolongados.
- ✓ Trabajos por lotes (procesos batch), ya que una transacción no debe durar más allá de tres segundos tanto por la necesidad de obtener respuestas rápidas, como por la conveniencia de no monopolizar recursos críticos del sistema en general.

Se pueden diseñar múltiples transacciones simples para simular una transacción compuesta (transacciones anidadas o encadenadas), lo cual implica:

- ✓ Mayor carga de trabajo en la etapa de programación.
- ✓ Mayor flexibilidad

1.2.10.2 Características de los sistemas transaccionales

A continuación se enumeran las características principales de los sistemas transaccionales:

- ✓ A través de éstos suelen lograrse ahorros significativos de mano de obra, debido a que automatizan tareas operativas de la organización.
- ✓ Con frecuencia son el primer tipo de Sistemas de Información que se implanta en las organizaciones. Se empieza apoyando las tareas a nivel operativo de la organización para continuar con los mandos intermedios y posteriormente con la alta administración conforme evolucionan.
- ✓ Son intensivos en entrada y salida de información; sus cálculos y procesos suelen ser simples y poco sofisticados. Estos sistemas requieren mucho manejo de datos para poder realizar sus operaciones y como resultado generan también grandes volúmenes de información.
- ✓ Tienen la propiedad de ser recolectores de información, es decir, a través de estos sistemas se cargan las grandes bases de información para su explotación posterior. Estos sistemas son los encargados de integrar gran cantidad de la información que se maneja en la organización, la cual será utilizada posteriormente para apoyar a los mandos intermedios y altos.
- ✓ Son fáciles de justificar ante la dirección general, ya que sus beneficios son visibles y palpables. El proceso de justificación puede realizarse enfrentando ingresos y costos. Esto se debe a que en el corto plazo se pueden evaluar los resultados y las ventajas que se derivan del uso de este tipo de sistemas. Entre las ventajas que pueden medirse se encuentra el ahorro de trabajo manual.
- ✓ Son fácilmente adaptables a paquetes de aplicación que se encuentran en el mercado, ya que automatizan los procesos básicos que por lo general son similares o iguales en otras organizaciones. Ejemplos de este tipo de sistemas son la facturación, nóminas, cuentas por cobrar, cuentas por pagar, contabilidad general, conciliaciones bancarias, inventarios, etcétera.
- ✓ Los sistemas transaccionales no sólo colaboran en la captura de la información transaccional, también ayudan a que las transacciones se realicen (e incluso llegan a automatizarlas) controlando los flujos de información y datos necesarios y, en su caso, emitiendo la documentación necesaria.

2 El Lenguaje de Modelado Unificado (UML)

UML es un Lenguaje gráfico que permite **modelar, construir y documentar** los elementos que forman un sistema software orientado a objetos.

No define un proceso de desarrollo del software. No es una metodología en sí mismo, sino un Lenguaje de modelización independiente de la metodología de desarrollo elegida (p.e., RUP y Métrica3).

Inicialmente concebido por Rumbaugh, Booch y Jacobson. Actualmente en el momento de elaborar este temario, la última versión es la 2.5. UML es un estándar del OMG (*Object Management Group*).

En general, resta protagonismo al diagrama de clases. Mejora la descripción dinámica del sistema mediante diagramas como el de transición de estados y los de interacción.

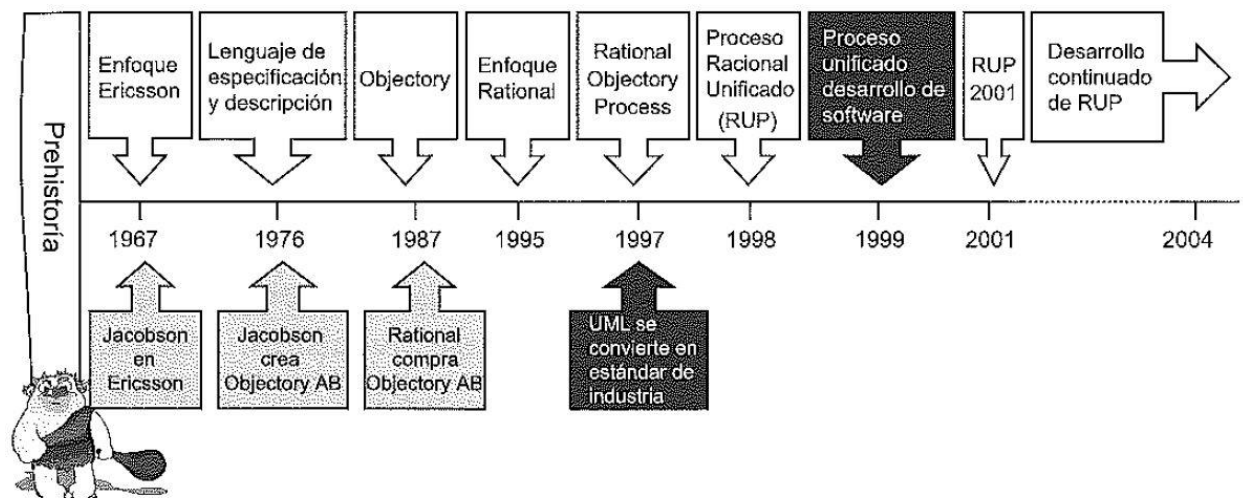


Figura 21 .- Evolución de los lenguajes de modelado de objetos hasta el lanzamiento de UML

2.1 Evolución de UML desde otras Metodologías/Notaciones OO

UML ha ido evolucionando desde la segunda mitad de la década de 1990 y tiene sus raíces en los métodos orientados a objetos desarrollado a finales de 1980 y principios de 1990. La línea de tiempo de la figura anterior muestra lo más destacado de la historia de los métodos de modelado orientado a objetos y la notación.

Después de que Rational Software Corporation contratara a James Rumbaugh, de General Electric en 1994, la compañía se convirtió en la fuente de dos de los métodos de modelización orientados a objetos más populares de la época: La técnica de modelado de objetos de Rumbaugh (OMT) y el método de Grady Booch, conocido como Diseño orientado a objetos (OOD).

Sus esfuerzos fueron complementados por Ivar Jacobson, el creador del método orientado a objetos de ingeniería de software (OOSE). Jacobson se unió a Rational en 1995, después de que su compañía, Objectory AB, fuera adquirida por Rational. A los tres metodólogos se les conoce normalmente como los Tres Amigos.

En 1996, Rational concluyó que la abundancia de lenguajes de modelado estaba retardando la adopción de la tecnología de objetos, reorientó el trabajo hacia un método unificado que encargó a los tres amigos con el desarrollo de un lenguaje de modelado unificado no propietario. Los representantes de las empresas del sector fueron consultados durante el OOPSLA '96 sobre dicho Lenguaje. Estos eligieron cajas para representar clases en lugar de los símbolos de nubes que se utilizaba en la notación de Booch.

Bajo la dirección técnica de los tres amigos, se organizó en 1996 un consorcio internacional llamado Consorcio UML (UML Partners) para completar las especificaciones del Lenguaje de Modelado Unificado (UML), y proponerlo como respuesta a la solicitud de propuesta (RFP) del OMG. El borrador de la especificación 1.0 del Consorcio UML se propuso al OMG en enero de 1997. Durante el mismo mes, el Consorcio UML formó un Grupo de Trabajo Semántico, (SemanticsTaskForce) presidido por Cris Kobryn y administrado por Ed Eykholt, para finalizar la semántica de la especificación e integrarlo con otros esfuerzos de estandarización. El resultado de este trabajo fue el UML 1.1, que se remitió al OMG en agosto de 1997 y fue aprobado en noviembre de 1997.

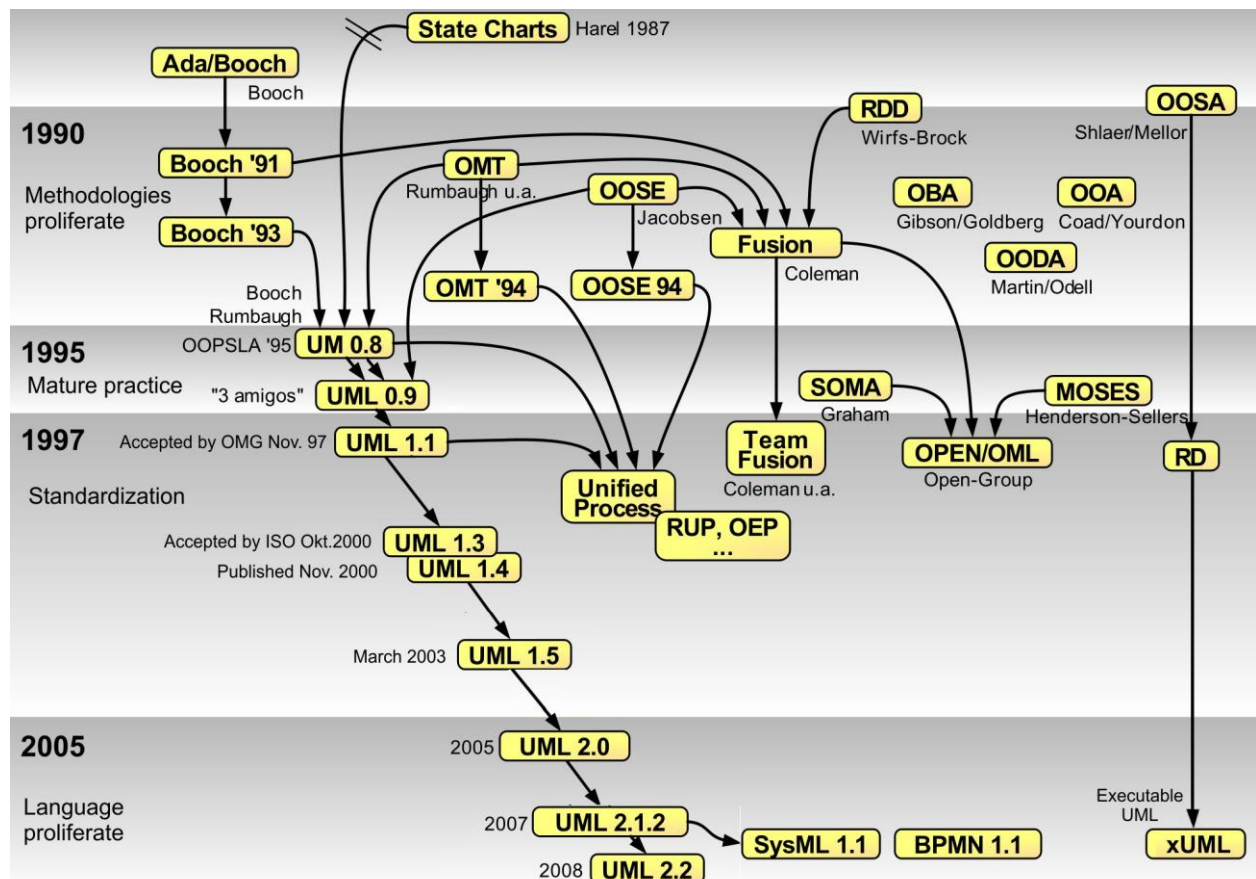


Figura 22.- Evolución de los distintos lenguajes y técnicas de modelado desde Booch a UML 2.X

2.2 UML 1.X

Como notación de modelado, la influencia de la notación OMT es predominante (por ejemplo, en el uso de rectángulos para representar clases y objetos). Aunque la notación en "nube" de Booch se ha caído, la capacidad de Booch para especificar los detalles del diseño de bajo nivel se adoptaron. La notación de casos de uso de Objectory y la notación de componente de Booch se integran con el resto de la notación, pero la integración semántica era relativamente débil en UML 1.1, y no se fijan realmente hasta la revisión mayor UML 2.0.

Conceptos de muchos otros métodos OO también estaban débilmente contemplados en UML, con la intención de que UML soportase todos los métodos OO. Muchos otros autores también contribuyeron, con sus distintas aproximaciones de los muchos modelos de la época, entre ellos: Tony Wasserman y Peter Pircher con la notación "Object-Oriented Structured Design (OOSD)" (no es un método), Ray Buhr con su "Diseño de Sistemas con Ada", "casos de uso y análisis de tiempo" de Archie Bowen, análisis de datos de Paul Ward y los diagramas de Estado "Statecharts" de David Harel. El grupo trató de asegurar una amplia cobertura en el campo de sistemas de tiempo real. Como resultado, UML es útil en una amplia variedad de problemas de ingeniería, desde procesos individuales, aplicaciones

monousuario hasta sistemas concurrentes y distribuidos, haciendo UML no sólo completo sino también grande.

El Lenguaje de Modelado Unificado es un estándar internacional: ISO / IEC 19501:2005 Tecnología de la información - Procesamiento distribuido abierto - Lenguaje de Modelado Unificado (UML) Version 1.4.2.

2.3 UML 2.X

UML ha madurado considerablemente desde UML 1.1. Varias revisiones menores (UML 1.3, 1.4 y 1.5) han corregido defectos y errores de la primera versión de UML. A éstas le ha seguido la revisión mayor UML 2.0 que fue adoptada por el OMG en 2005.

Aunque UML 2.1 nunca fue lanzado como una especificación formal, las versiones 2.1.1 y 2.1.2, aparecieron en 2007, seguidas por UML 2.2 en febrero de 2009. UML 2.3 fue lanzado oficialmente en mayo de 2010. UML 2.4.1 fue lanzado oficialmente en agosto de 2011.

UML 2.5 fue formalmente liberada en junio de 2015, y se considera una versión con cambios menores respecto a la 2.4.1.

2.3.1 Estándares que conforman UML 2.X

UML está compuesto por los siguientes “Estándares”:

- ✓ **Superestructura:** Es la especificación que usamos todos los días. Aquí se encuentran todos los diagramas que la mayoría de los desarrolladores conocen. La versión actual es la “**UML Superstructureversion 2.4.1**”.
- ✓ **Infraestructura:** Conceptos de bajo nivel. Meta-Modelo da soporte a la superestructura, entre otras. La última versión es la “**UML Infraestructureversion 2.4.1**”. **Ha dejado de formar parte de los estándares conjuntos.**
- ✓ **OCL:** Lenguaje de restricción de Objetos. De utilidad para especificar conceptos ambiguos sobre los distintos elementos del diagrama. La versión actual es la “**OCL version 2.3.1**”.
- ✓ **XMI/Intercambio de diagramas:** Permite compartir diagramas entre herramientas de modelado UML. La versión actual es la “**UML DiagramInterchangeversion 1.1**”.

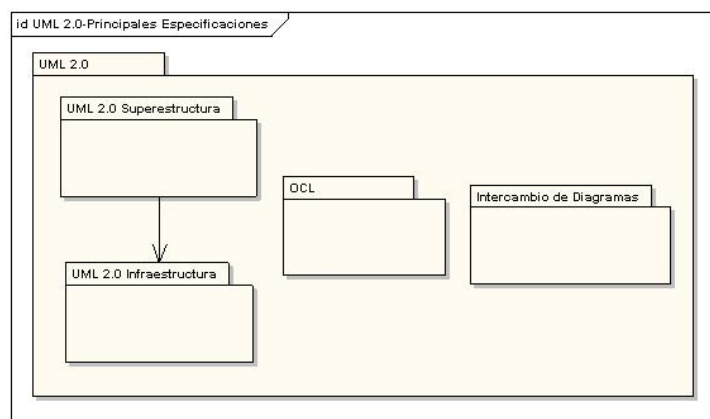


Figura 23 .- Estándares que conforman UML 2.X

UML posee varios mecanismos de extensibilidad del lenguaje de modelado:

- ✓ **Estereotipo:** Un estereotipo permite indicar especificaciones del lenguaje al que se refiere el diagrama de UML (un paquete de estereotipo <<subsistema>> o una clase <<persistente>>).
- ✓ **Restricciones:** identifican un comportamiento forzado de una clase o relación. Mediante la restricción se está forzando el comportamiento del objeto al que se le aplica. **OCL.**
- ✓ **Un valor etiquetado** extiende las propiedades de un bloque de construcción UML, añade nueva información en la especificación de ese elemento (p.e. versión de un componente).

UML posee **elementos comunes a cualquier diagrama UML**:

- ✓ **Las notas**: sirven para añadir cualquier tipo de comentario a un diagrama o a un elemento de un diagrama.
- ✓ **Las dependencias**: entre dos elementos de un diagrama significa que un cambio en el elemento destino puede implicar un cambio en el elemento origen.

2.3.2 Tipos de diagramas (UML 2.5)

UML consta de varios tipos de diagramas, que se clasifican en “**Diagramas de Estructura**” y “**Diagramas de comportamiento**” para representar la visión estática y dinámica respectivamente.

Los diagramas de Estructura enfatizan en “los elementos que deben existir en el sistema modelado”:

- ✓ Diagrama de clases
- ✓ Diagrama de componentes
- ✓ Diagrama de objetos
- ✓ Diagrama de estructura compuesta (UML 2.0)
- ✓ Diagrama de despliegue
- ✓ Diagrama de paquetes
- ✓ Diagrama de Perfiles (Profile) (UML 2.5)

Los Diagramas de Comportamiento enfatizan en lo que debe suceder en el sistema modelado:

- ✓ Diagrama de actividades
- ✓ Diagrama de casos de uso
- ✓ Diagrama de estados
- ✓ Diagramas de Interacción

Los Diagramas de Interacción son un subtipo de diagramas de comportamiento, que enfatiza sobre el flujo de control y de datos entre los elementos del sistema modelado:

- ✓ Diagrama de secuencia
- ✓ Diagrama de comunicación → versión simplificada del Diagrama de colaboración
- ✓ Diagrama de tiempos (UML 2.0)
- ✓ Diagrama global de interacciones o Diagrama de vista de interacción (UML 2.0)

Estos diagramas se pueden ver en la siguiente figura.

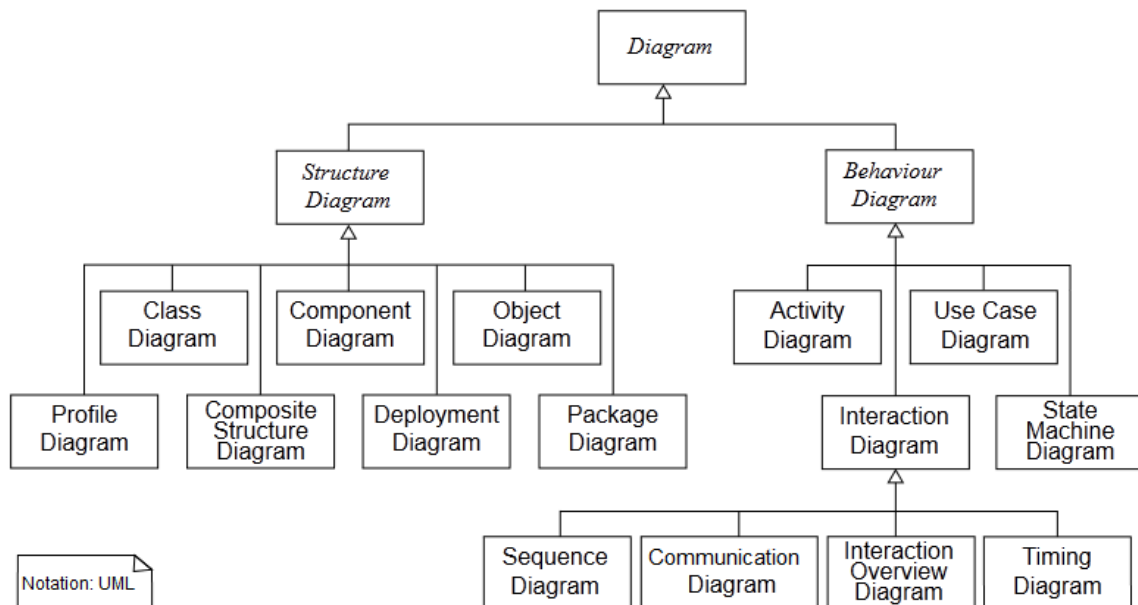


Figura 24 .- Diagramas UML 2.5

2.3.2.1 Descripción de los Diagramas de UML

La siguiente tabla muestra una breve descripción de los mismos y la prioridad que se le asigna al mismo dentro del modelado en relación con el resto de diagramas. Esto no indica que en un sistema concreto deba priorizarse el uso de un diagrama frente a otro, ya que dependerá de las características concretas del sistema, y los aspectos que se quieran enfatizar y detallar mejor. No todos los diagramas son necesarios para modelar todos los sistemas.

Diagrama	Descripción	Prioridad
Diagrama de Clases	Muestra una colección de elementos de modelado declarativo (estáticos), tales como clases, tipos y sus contenidos y relaciones.	Alta
Diagrama de Componentes	Representa los componentes que componen una aplicación, sistema o empresa. Los componentes, sus relaciones, interacciones y sus interfaces públicas.	Media
Diagrama de Estructura de Composición	Representa la estructura interna de un clasificador (tal como una clase, un componente o un caso de uso), incluyendo los puntos de interacción de clasificador con otras partes del sistema.	Baja
Diagrama de Despliegue Físico	Un diagrama de despliegue físico muestra cómo y dónde se desplegará el sistema. Las máquinas físicas y los procesadores se representan como nodos y la construcción interna puede ser representada por nodos o artefactos embebidos. Como los artefactos se ubican en los nodos para modelar el despliegue del sistema, la ubicación es guiada por el uso de las especificaciones de despliegue.	Media
Diagrama de Objetos	Un diagrama que presenta los objetos y sus relaciones en un punto del tiempo. Un diagrama de objetos se puede considerar como un caso especial de un diagrama de clases o un diagrama de comunicaciones.	Baja
Diagrama de Paquetes	Un diagrama que presenta cómo se organizan los elementos de modelado en paquetes y las dependencias entre ellos, incluyendo importaciones y extensiones de paquetes.	Baja
Diagrama de Actividades	Representa los procesos de negocios de alto nivel, incluidos el flujo de datos. También puede utilizarse para modelar lógica compleja y/o paralela dentro de un sistema.	Alta
Diagrama de Comunicaciones (anteriormente: Diagrama de colaboraciones)	Es un diagrama que enfoca la interacción entre líneas de vida, donde es central la arquitectura de la estructura interna y cómo ella se corresponde con el pasaje de mensajes. La secuencia de los mensajes se da a través de un esquema de numerado de la secuencia.	Baja
Diagrama de Revisión de la Interacción	Los Diagramas de Revisión de la Interacción enfocan la revisión del flujo de control, donde los nodos son Interacciones u Ocurrencias de Interacciones. Las Líneas de Vida de los Mensajes no aparecen en este nivel de revisión.	Baja
Diagrama de Secuencias	Un diagrama que representa una interacción, poniendo el foco en la secuencia de los mensajes que se intercambian, junto con sus correspondientes ocurrencias de eventos en las Líneas de Vida.	Alta

Diagrama	Descripción	Prioridad
Diagrama de Máquinas de Estado	Un diagrama de Máquina de Estados ilustra cómo un elemento, muchas veces una clase, se puede mover entre estados que clasifican su comportamiento, de acuerdo con disparadores de transiciones, guardias de restricciones y otros aspectos de los diagramas de Máquinas de Estados, que representan y explican el movimiento y el comportamiento.	Media
Diagrama de Tiempos	El propósito primario del diagrama de tiempos es mostrar los cambios en el estado o la condición de una línea de vida (representando una Instancia de un Clasificador o un Rol de un clasificador) a lo largo del tiempo lineal. El uso más común es mostrar el cambio de estado de un objeto a lo largo del tiempo, en respuesta a los eventos o estímulos aceptados. Los eventos que se reciben se anotan, a medida que muestran cuándo se desea mostrar el evento que causa el cambio en la condición o en el estado.	Baja
Diagrama de Casos de Uso	Un diagrama que muestra las relaciones entre los actores y el sujeto (sistema), y los casos de uso.	Alta
Diagrama de Perfil (Profile)	Un Perfil se define en un paquete UML, estereotipado <<profile>>, que extiende a un metamodelo o a otro Perfil. El paquete contendrá las extensiones del meta-modelo. Hay tres mecanismos que se utilizan para definir Perfiles: ✓ Estereotipos (stereotypes) ✓ Valores etiquetados (tagged values) ✓ Restricciones (constraints)	Baja

2.3.3 Arquitectura de Sistemas

"The UML Reference Manual" define la arquitectura de sistema como **"La estructura organizacional de un sistema, incluida su descomposición en partes, su conectividad, interacción, mecanismos y los principios directores que informan al diseño de un sistema"**.

IEEE define la arquitectura de sistema como **"el concepto de más alto nivel de un sistema en su entorno"**, La arquitectura trata sobre capturar los aspectos estratégicos de la estructura de alto nivel de un sistema.

Existen muchas formas de examinar la arquitectura, pero una forma muy común es la de "Vistas 4+1" descrito por Phillippe Kruchten. Los aspectos esenciales de la arquitectura del sistema se capturan en cuatro vistas diferentes:

- ✓ Vista de Lógica (diseño): Diagramas de Clases, Objetos, Interacción, Estados y Actividades.
- ✓ Vista de implementación: Gestión de configuración. Diagrama de componentes.
- ✓ Vista de procesos: Sincronización, concurrencia y rendimiento. Clases activas.
- ✓ Vista de despliegue: Diagramas de despliegue.

Todas estas vistas están integradas por una quinta vista, la vista de caso de uso: Diagramas de casos de uso.

Cada una de estas vistas aborda diferentes aspectos de la arquitectura de software como se indica en la figura.

- ✓ **Vista lógica:** Captura el vocabulario del ámbito del problema como un conjunto de clases y objetos. El énfasis está en mostrar cómo los objetos y las clases que componen un sistema implementan el comportamiento requerido del sistema.
- ✓ **Vista de proceso:** Modela los procesos en un sistema como clases activas. Es una variación orientada al proceso de la vista lógica y contiene los mismos artefactos.
- ✓ **Vista de implementación:** Modela los archivos y componentes sobre la base de código físico del sistema. También trata sobre ilustrar dependencias entre componentes y sobre la

gestión de la configuración de conjuntos de componentes para definir una versión del sistema.

- ✓ **Vista de despliegue:** Modela el despliegue físico de artefactos en un conjunto de nodos físicos computacionales como ordenadores y periféricos. Le permite modelar la distribución de artefactos en los nodos de un sistema distribuido.
- ✓ **Vista de caso de uso:** Captura los requisitos básicos para el sistema como un conjunto de casos de uso. Estos casos de uso proporcionan la base para la construcción de otras vistas.

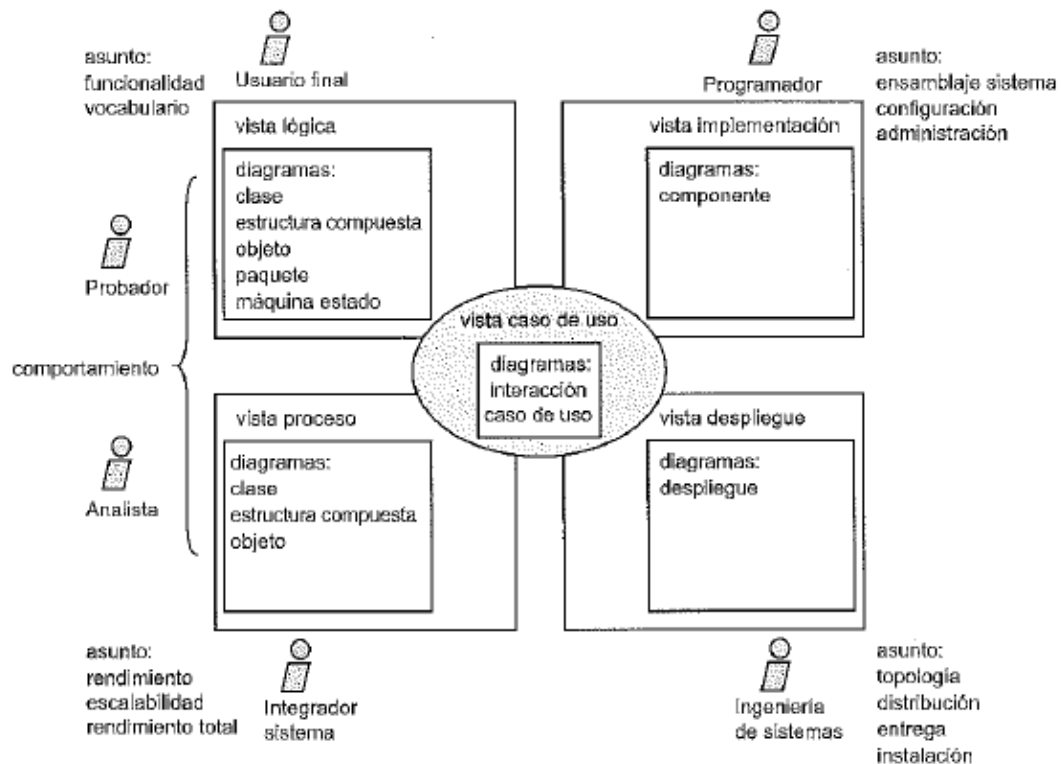


Figura 25 .- Modelo de arquitectura de sistema mediante Vistas 4+1" descrito por Philippe Kruchten

Este diagrama se utiliza para la captura de requisitos funcionales del usuario. Guía de todo el proceso de desarrollo del sistema de información.

2.4 Diagrama de Despliegue

El diagrama de despliegue, como se ha comentado anteriormente es un tipo de diagrama estructural. La mayoría de las veces el modelado de la vista de despliegue implica modelar la topología del hardware sobre el que se ejecuta el sistema. Aunque UML no es un lenguaje de especificación hardware de propósito general, se ha diseñado para modelar muchos de los aspectos hardware de un sistema a un nivel suficiente para que un ingeniero software pueda especificar la plataforma sobre la que se ejecuta el software del sistema, mostrando por tanto las relaciones físicas entre los componentes software y hardware del sistema. En él se indica la situación física de los componentes lógicos desarrollados, es decir, se sitúa el software en el hardware que lo contiene. Cada hardware se representa como un nodo.

Algunos de los usos que se les da a los diagramas de despliegue son para modelar:

- ✓ **Sistemas empotrados:** Un sistema empotrado es una colección de hardware con una gran cantidad de software que interactúa con el mundo físico.
- ✓ **Sistemas cliente-servidor:** Los sistemas cliente-servidor son un extremo del espectro de los sistemas distribuidos y requieren tomar decisiones sobre la conectividad de red de los clientes a los servidores y sobre la distribución física de los componentes software del sistema a través de nodos.

- ✓ **Sistemas completamente distribuidos:** En el otro extremo encontramos aquellos sistemas que son ampliamente o totalmente distribuidos y que normalmente incluyen varios niveles de servidores. Estos sistemas contienen a menudo varias versiones de componentes software, alguno de los cuales pueden incluso migrar de un nodo a otro, requiriendo su diseño tomar decisiones que permitan un cambio continuo de la topología del sistema.

Los elementos usados por este tipo de diagrama son nodos (puede haber instancias y Estereotipos), componentes (representados como una caja rectangular con dos protuberancias del lado izquierdo) y asociaciones.

Nodo: Un nodo es un elemento donde se ejecutan los componentes, y representa el despliegue físico de estos componentes. Gráficamente, un nodo se representa como un cubo. Puede representar tanto hardware como software.

Instancia de Nodo: Una instancia de nodo se puede mostrar en un diagrama. Una instancia se puede distinguir desde un nodo por el hecho de que su nombre está subrayado y tiene dos puntos antes del tipo de nodo base. Una instancia puede o no tener un nombre antes de los dos puntos. El siguiente diagrama muestra una instancia nombrada de un Ordenador o servidor.

Estereotipo de Nodo: se proveen distintos tipos de estereotipos estándar para los nodos, nombrados «cdrom», «cd-rom», «computer», «disk array», «pc», «pc client», «pc server», «secure», «server», «storage», «unix server», «user pc». Estos mostrarán un icono apropiado en la esquina derecha arriba del símbolo nodo.

Artefacto: Un artefacto es un producto del proceso de desarrollo de software, que puede incluir los modelos del proceso (e.g. modelos de Casos de Uso, modelos de Diseño, etc.), archivos fuente, ejecutables, documentos de diseño, reportes de prueba, prototipos, manuales de usuario y más.

Un artefacto se denota por un rectángulo mostrando el nombre del artefacto, el estereotipo «artifact» y un icono de documento, como a continuación.

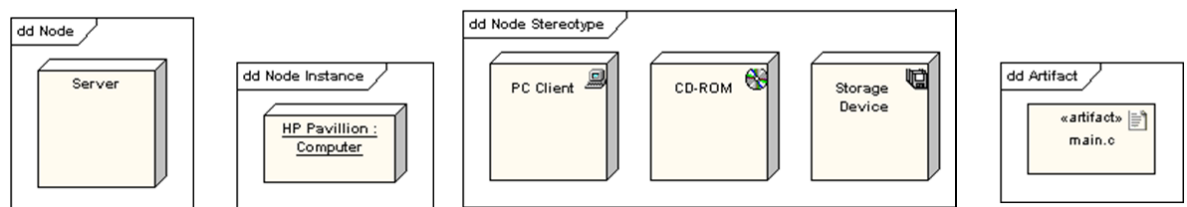


Figura 26 .- Nodo, Instancia, estereotipos y artefactos

Asociación: En el contexto del diagrama de despliegue, una asociación representa una ruta de comunicación entre los nodos. El siguiente diagrama muestra un diagrama de despliegue para una red, mostrando los protocolos de red como estereotipos.

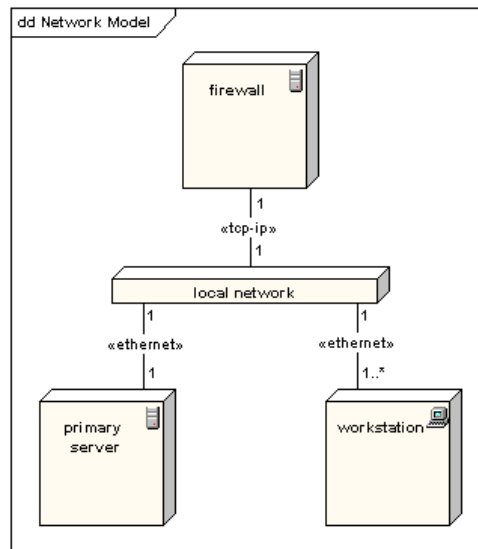


Figura 27 .- Nodo, Instancia, estereotipos y artefactos

Se pueden mostrar las multiplicidades en los extremos de la asociación si es necesario.

Nodo como contenedor:

Un nodo puede contener otros elementos, como componentes o artefactos.

En la siguiente figura se tienen dos nodos, el cliente y el servidor. Cada uno de ellos contiene componentes. El componente del cliente utiliza un interface de uno de los componentes del servidor. Se muestra la relación existente entre los dos nodos. A esta relación se le podría asociar un estereotipo para indicar el tipo de conexión de que se dispone entre el cliente y el servidor.

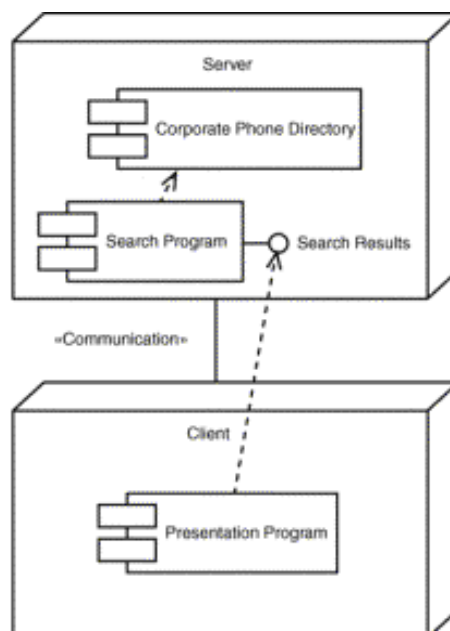


Figura 28 .- Diagrama de Despliegue Cliente-Servidor

Como los componentes pueden residir en más de un nodo, se puede situar el componente de forma independiente, sin que pertenezca a ningún nodo, y relacionarlo con los nodos en los que se sitúa.

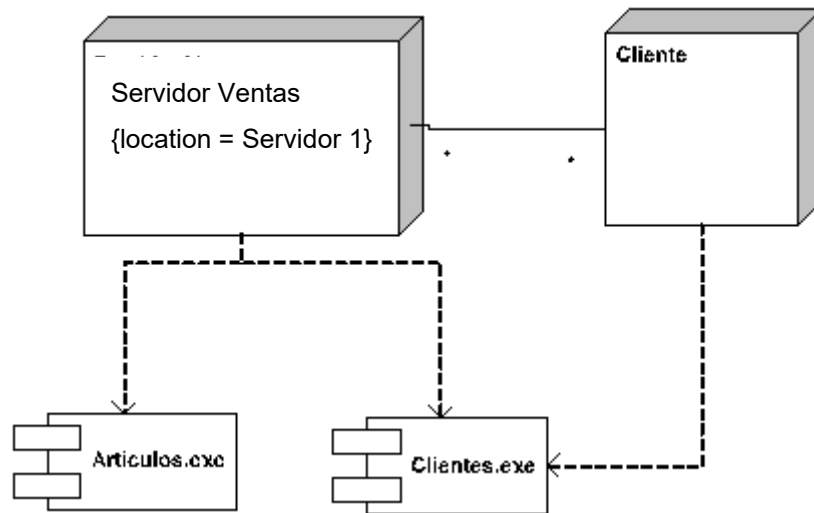


Figura 29 Diagrama de Despliegue con componentes distribuidos

El siguiente diagrama muestra un diagrama de despliegue para una parte del sistema embebido y muestra un artefacto ejecutable como contenido por el nodo madre (motherboard).

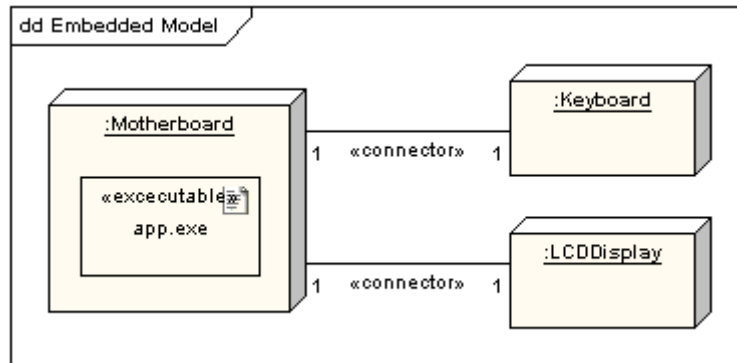


Figura 30 .- Diagrama de despliegue parcial de un sistema embebido

A continuación se muestra un diagrama de despliegue de un sistema completo con servidores de aplicaciones del negocio, y externos dedicados a tareas de autenticación y autorización, servidores LDAP y de BBDD y servidores de documentación.

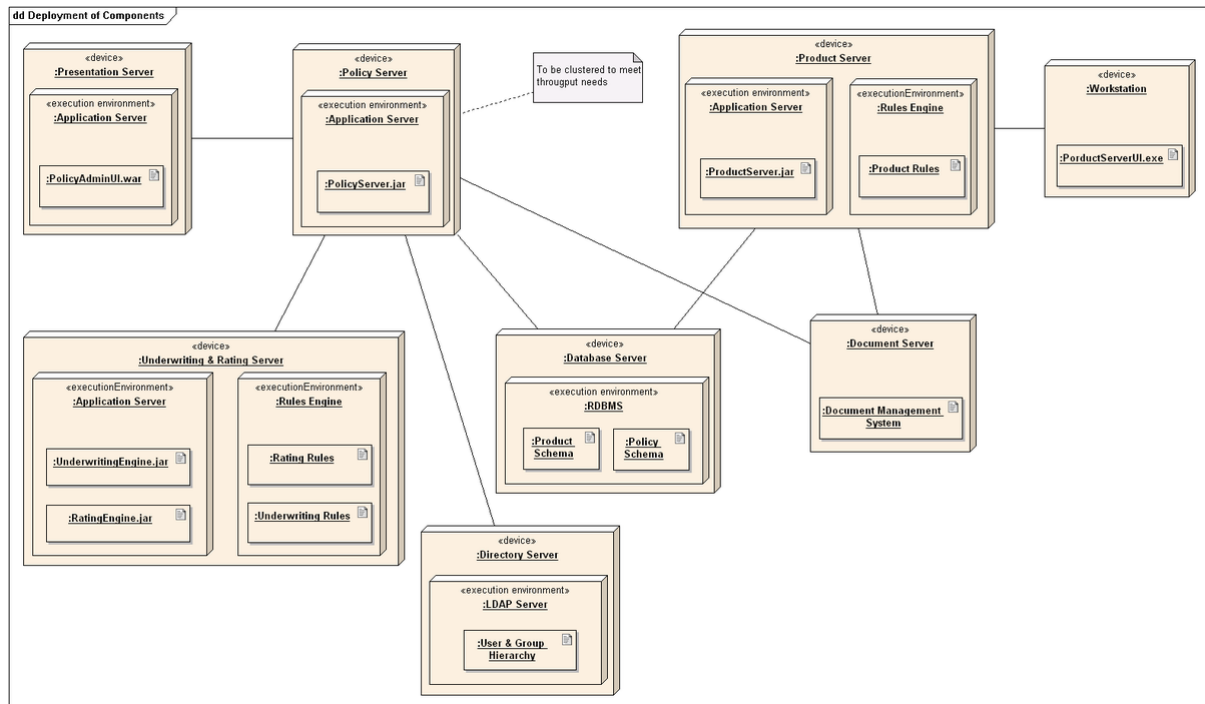


Figura 31 .- Diagrama de despliegue de un sistema completo

3 Glosario

CASE: Computer Aided Software Engineering

CORBA: Common Object Request Broker Architecture

E/R: Entidad Relación

DEC: Diagrama de Estructura de Cuadros de Constantine

DFD: Diagrama de Flujo de datos

DSI: Diseño del Sistema de Información

OCL: Lenguaje de restricción de Objetos

XMI: Intercambio de diagramas

OMA: Object Management Architecture

OMG: Object Management Group

OO: Orientación a Objetos

OOD: Diseño orientado a objetos

OMT: Object Modelling Technique

OSF: Open Source Foundation

RUP: Rational Unified Process

SGBDR: Sistema de Gestión de Base de Datos Relacional

TAD: Tipos Abstractos de Datos

UML: Unified Modelling Language

CONCLUSIONES

El tema de diseño arquitectónico ha introducido los distintos tipos de diseño (Arquitectónico o alto nivel y detallado o bajo nivel) tratando de reflejar los principales estilos arquitectónicos que se pueden emplear en el diseño de sistemas software. Muchos de estos estilos son tratados en mayor profundidad por otros temas del temario por lo que se ha preferido referenciar a los mismos para evitar incoherencias y facilitar la tarea del opositor.

Igualmente se han incluido referencias a contenido como la Orientación a objetos o UML por no estar explícitamente contemplados en ningún otro tema del temario y ser necesarios para entender alguno de los puntos que debe desarrollar este tema como es el caso de los diagramas de despliegue (UML) o la arquitectura orientada a Objetos (que requieren al menos un conocimiento general del paradigma orientado a Objetos).

Estilos de Arquitectura de sistemas:

Arquitectura centrada en datos

- ✓ **Arquitectura de pizarra**
- ✓ **Data Warehouse**

Arquitectura de flujo de datos

- ✓ **Transformación por lotes**
- ✓ **Transformación continua** - Tuberías y filtros.

Arquitecturas de llamada y retorno:

- ✓ Interfaz interactiva
- ✓ **Arquitecturas estratificadas o por capas o niveles.**
- ✓ **MVC - Modelo Vista Controlador**

Arquitecturas orientadas a objetos

Arquitecturas orientadas a aspecto

Arquitecturas orientadas a servicios (SOA)

Arquitecturas de Sistemas Especiales:

- ✓ **Sistema en tiempo real**
- ✓ **Sistemas transaccionales**

Tareas a realizar en la fase de diseño

Alto nivel:

- Selección de la arquitectura global del sistema
- Descomposición de subsistemas en sistemas
- Identificación de concurrencias
- Control de los recursos compartidos
- Gestión de los datos
- Diseño de las interfaces

Bajo Nivel

- Identificación y manejo de excepciones
- Selección de los algoritmos y de las estructuras de datos.
- Preparación para la migración y carga inicial de datos.
- Especificación técnica de las pruebas

EL LENGUAJE DE MODELADO UNIFICADO (UML)

Evolución de UML desde otras Metodologías/Notaciones OO

UML 1.X

UML 2.X

Estándares que conforman UML 2.X

Superestructura

Infraestructura: Conceptos de bajo nivel. Meta-Modelo

OCL: Lenguaje de restricción de Objetos

XMI/Intercambio de diagramas:

Descripción de los Diagramas de UML

Arquitectura de Sistemas: Modelo vistas 4+1

Vista de Lógica (diseño): Diagramas de Clases, Objetos, Interacción, Estados y Actividades.

Vista de implementación: Gestión de configuración. Diagrama de componentes.

Vista de procesos: Sincronización, concurrencia y rendimiento. Clases activas.

Vista de despliegue: Diagramas de despliegue.

Vista de caso de uso: Diagramas de casos de uso.

Diagramas UML 2.5: (El resto de diagramas se tratan en otros temas del temario)

Diagrama de despliegue

PREGUNTAS DE TEST

- 1) **De las siguientes afirmaciones en relación con el diseño del software, indique cuál de ellas es verdadera :**
 - a) Una característica de un buen diseño es que el acoplamiento entre módulos sea lo más bajo posible.
 - b) Cuantas más conexiones existan entre dos módulos, más bajo es el acoplamiento entre los mismos.
 - c) De los distintos tipos de acoplamiento, el acoplamiento por contenido es el que garantiza la mejor calidad de diseño.
 - d) Son ciertas todas las premisas anteriores.

- 2) **El Análisis de Transacción :**
 - a) Es una modalidad dentro de las técnicas de análisis orientadas al flujo de datos.
 - b) Es una estrategia de diseño para pasar de los Diagramas de Flujo de Datos a los Diagramas de Estructura de Cuadros, que se aplica cuando el DFD toma una forma en la que un dato determina la elección de uno o más flujos de información.
 - c) Es una técnica para la evaluación y el refinamiento del diseño.
 - d) Son falsas todas las premisas anteriores.

- 3) **Dentro de la Fase de Diseño se pueden considerar las siguientes partes :**
 - a) Diseño funcional, diseño orgánico, diseño técnico y diseño detallado.
 - b) Diseño arquitectónico, diseño de datos y diseño procedimental.
 - c) Diseño conceptual y diseño detallado.
 - d) Diseño estructurado, diseño de datos, diseño de procesos y diseño técnico.

- 4) **Una Arquitectura Software se define mediante tres elementos. señale los correctos:**
 - a) procesamiento, datos y conexión
 - b) elementos, procesamiento y conexión
 - c) elementos, forma y razón.
 - d) elementos, datos y conexión

- 5) **En relación a las tareas de diseño de “alto nivel”, señale la que no lo sea:**
 - a) Selección de la arquitectura global del sistema
 - b) Especificación técnica de las pruebas.
 - c) Descomposición en subsistemas de sistemas
 - d) Identificación de concurrencias

- 6) **En relación a las tareas de diseño de “bajo nivel”, señale la que lo sea:**
 - a) Identificación y manejo de excepciones
 - b) Selección de los algoritmos y de las estructuras de datos
 - c) Preparación para la migración y carga inicial de datos
 - d) Todas las anteriores

- 7) **En relación a la Arquitectura en pizarra, señale de los siguientes componentes el que no lo sea:**
- a) Fuente de conocimiento
 - b) Pizarra
 - c) Intérprete de comandos de control
 - d) Todos los anteriores
- 8) **Entre las principales ventajas del estilo arquitectónico de Tuberías y filtros se encuentran:**
- a) Fuerza un procesamiento secuencial aunque permite una ejecución concurrente
 - b) Facilidad de mantenimiento y mejora
 - c) Facilidad de diagnóstico en el caso de errores
 - d) todas las anteriores
- 9) **Entre las principales ventajas del estilo arquitectónico de Tuberías y filtros se encuentran:**
- a) Es muy adecuado cuando hay fuertes restricciones de rendimiento
 - b) Fuerza un procesamiento secuencial aunque permite una ejecución concurrente
 - c) Es muy apto para situaciones interactivas
 - d) Todas las anteriores
- 10) **En relación a las características de las Arquitecturas de Objetos Distribuidas señale la que no lo sea:**
- a) Sencillez para probar
 - b) Concurrencia
 - c) Independencia ante Fallos o Mayor resistencia ante fallos
 - d) Todas las anteriores lo son
- 11) **Los diagramas de despliegue no son adecuados para modelar**
- a) Sistemas empotrados
 - b) Sistemas cliente-servidor
 - c) Sistemas completamente distribuidos
 - d) Es adecuado para modelar cualquier sistema
- 12) **El modelo MVC - Modelo Vista Controlador se corresponde con un estilo de**
- a) Arquitectura centrada en datos
 - b) Arquitectura de flujo de datos
 - c) Arquitecturas de llamada y retorno
 - d) Arquitecturas orientadas a objetos

13) En relación a UML y al modelo arquitectónico de vistas 4+1, en relación a la Vista de despliegue señale la afirmación correcta:

- a) Modela el despliegue físico de artefactos en un conjunto de nodos físicos computacionales como ordenadores y periféricos. Le permite modelar la distribución de artefactos en los nodos de un sistema distribuido.
- b) Modela los archivos y componentes sobre la base de código físico del sistema.
- c) Ilustra las dependencias entre componentes y sobre la gestión de la configuración de conjuntos de componentes para definir una versión del sistema.
- d) Todas las anteriores son correctas

14) El modelo de procesamiento Batch se corresponde con un estilo arquitectónico de

- a) Arquitectura de flujo de datos
- b) Arquitecturas orientadas a objetos
- c) Arquitectura centrada en datos
- d) Arquitecturas de llamada y retorno

15) En relación a la Arquitectura centrada en datos, se poseen varios tipos de repositorio, señale el que lo sea:

- a) Repositorio pasivo
- b) Repositorio pizarrón
- c) B y C los son
- d) Ninguna de las anteriores

SOLUCIONES A LAS PREGUNTAS DE TEST

PREGUNTA	SOLUCIÓN
1	A
2	B
3	B
4	C
5	B
6	D
7	D
8	D
9	B
10	A
11	D
12	C
13	A
14	A
15	C

BIBLIOGRAFÍA BÁSICA

1. Sommerville, Ian. Ingeniería del Software. Ed. Addison Wesley. 9ª Ed. 2011. ISBN: 9780137035151
2. <http://www.tutorialspoint.com/uml/>
3. <http://www.sparxsystems.com/uml-tutorial.html>
4. Metodología Métrica versión 3. Guía de Técnicas y prácticas. Ministerio de Administraciones Públicas 1998.
http://administracionelectronica.gob.es/pae_Home/pae_Documentacion/pae_Metodolog/pae_Metrica_v3.html
5. <http://uml.org>
6. https://es.wikipedia.org/wiki/Sistema_de_tiempo_real
7. es.wikipedia.org/wiki/Lenguaje_unificado_de_modelado
8. <https://www.visual-paradigm.com/tutorials/>
9. Pressman, Roger. Ingeniería del Software. Un Enfoque Práctico. Ed. McGraw Hill. 6ª Ed. 2005. ISBN 970-105473-3
10. <http://www.exa.unicen.edu.ar/catedras/prog1/introprog2/sites/default/files/ApuntesDiagramaEstructura.pdf>

BIBLIOGRAFÍA PARA AMPLIAR EL TEMA

1. <http://modeling-languages.com/herramientas-para-uml/>
2. UML Resource Center. Rational Software. <http://www.ibm.com/software/rational>
3. Object-Oriented Analysis and Design. G. Booch. Benjamin/Cummings
4. Análisis estructurado moderno.. Prentice. Yourdon, E
5. <http://www.angelfire.com/scifi/jzavalar/apuntes/IngSoftware.html>
6. <http://www.omg.org/spec/UML/Current>, actualmente redirige a la versión 2.5
7. <https://es.wikipedia.org/wiki/Modelo-vista-controlador>
8. El Lenguaje Unificado de Modelado, G. Booch, J. Rumbaugh, I. Jacobson. Addison Wesley
9. <http://www.uml-diagrams.org>